

Behavioral Detection of Malware in IoT Systems

A study guide covering Linux internals, firmware analysis, malware analysis, and machine learning for security

Contents

Introduction	5
1 Linux Internals & Operating System Theory	6
1.1 The User/Kernel Boundary	6
1.1.1 Why the boundary exists	6
1.1.2 Crossing the boundary: the syscall mechanism	7
1.1.3 What happens during a syscall, step by step	7
1.1.4 Why crossing the boundary is expensive	8
1.1.5 The vDSO: not all "syscalls" are syscalls	9
1.2 The Syscall Interface	10
1.2.1 Syscall numbers	10
1.2.2 The syscall ABI by architecture	10
1.2.3 The syscalls you'll see constantly	11
1.2.4 Libc wrappers vs raw syscalls	12
1.3 The Process Model	13
1.3.1 Tasks, processes, threads	13
1.3.2 PIDs and TIDs	13
1.3.3 fork, vfork, clone, exec	14
1.3.4 Process states	15
1.3.5 Process groups, sessions, and controlling terminals	16
1.3.6 Signals	16
1.3.7 The /proc filesystem	17
1.4 Memory Model	18
1.4.1 Virtual memory: the abstraction	18
1.4.2 The address space layout of a process	19
1.4.3 ASLR	19
1.4.4 mmap and mprotect	20
1.4.5 ELF binary format	21
1.4.6 Dynamic linking and ld.so	22
1.5 File Descriptors and "Everything is a File"	23
1.5.1 What a file descriptor actually is	23
1.5.2 Standard file descriptors	24
1.5.3 Manipulating the FD table	24
1.5.4 Why FDs matter for behavioral analysis	24
1.6 ptrace: The Classic Tracing Mechanism	25
1.6.1 What ptrace does	25
1.6.2 How strace works	25
1.6.3 Limitations of ptrace	26

1.7	eBPF: Modern Tracing Infrastructure	27
1.7.1	What eBPF is	27
1.7.2	The verifier: why this is safe	28
1.7.3	Hook types: where programs can attach	28
1.7.4	BPF maps: state across invocations	29
1.7.5	bpfftrace, bcc, libbpf: three ways to write eBPF tools	29
1.7.6	Why eBPF wins for behavioral monitoring	30
1.8	Foundational Papers for Syscall-Based Detection	31
1.8.1	Forrest et al. 1996: “A Sense of Self for Unix Processes”	31
1.8.2	Forrest, Warrender, Pearlmuter 1999: “Detecting Intrusions Using System Calls: Alternative Data Models”	32
1.9	Track 1 Self-Check Questions	33
2	IoT and Embedded Systems	34
2.1	What “IoT” Means in This Context	34
2.1.1	The spectrum of embedded systems	34
2.1.2	Why the security is bad: structural economics	34
2.2	Architectures	35
2.2.1	The architecture zoo	35
2.2.2	The cross-architecture problem	36
2.2.3	What you need to know about each architecture	37
2.3	Firmware Structure	38
2.3.1	What’s in a firmware blob	38
2.3.2	Bootloaders	38
2.3.3	Filesystem formats	39
2.3.4	Device Tree	40
2.3.5	NVRAM	40
2.4	IoT Userspace Conventions	40
2.4.1	BusyBox: the heart of small Linux userspaces	40
2.4.2	libc choices: glibc vs uClibc vs musl	41
2.4.3	Init systems	42
2.4.4	Web servers	42
2.4.5	Vendor daemons	43
2.5	The IoT Attack Surface	43
2.5.1	Default credentials	43
2.5.2	Internet-exposed services	44
2.5.3	Memory-corruption vulnerabilities in admin services	44
2.5.4	Outdated kernels and userspace	45
2.5.5	The supply-chain problem	45
2.6	The Firmware Emulation Problem	46
2.6.1	Why emulate?	46
2.6.2	Why emulation is hard	46
2.6.3	Two flavors of emulation	47
2.6.4	The history of firmware emulation tools	47
2.6.5	What’s lost in emulation	47
2.7	The Current Tooling Landscape	48
2.7.1	Extraction	48
2.7.2	Reverse engineering	48
2.7.3	Dynamic analysis frameworks	49
2.7.4	Where to get firmware samples	49
2.8	Track 2 Self-Check Questions	50

3	Malware Analysis as a Discipline	52
3.1	The Taxonomy of Malware Analysis	52
3.1.1	Static vs dynamic vs hybrid	52
3.1.2	Where syscall-based detection fits	53
3.2	The Classic Malware Analysis Pipeline	55
3.2.1	Triage	55
3.2.2	Unpacking	55
3.2.3	Static analysis pass	55
3.2.4	Dynamic analysis pass	56
3.2.5	Family clustering	56
3.2.6	Reporting and IoCs	57
3.3	Static Analysis Features	58
3.3.1	Strings	58
3.3.2	Imports and dynamic symbols	58
3.3.3	Opcode n-grams	59
3.3.4	Control flow graph features	59
3.3.5	ELF metadata	59
3.3.6	Function-level signatures	60
3.4	Dynamic Analysis Features	60
3.4.1	Syscall sequences and frequencies	60
3.4.2	Network behavior	61
3.4.3	Filesystem activity	62
3.4.4	Process activity	62
3.4.5	Memory patterns	62
3.4.6	Combining feature modalities	63
3.5	Evasion Techniques	63
3.5.1	Anti-debugging	63
3.5.2	Anti-VM	64
3.5.3	Anti-sandbox	64
3.5.4	Packing and obfuscation	65
3.5.5	Polymorphism and metamorphism	66
3.5.6	Why IoT malware uses fewer evasion techniques	66
3.6	The IoT Malware Landscape	66
3.6.1	The pre-Mirai world	66
3.6.2	Mirai (2016): the watershed	67
3.6.3	Mirai variants and descendants	68
3.6.4	Gafgyt and parallel evolution	68
3.6.5	Hajime (2016)	68
3.6.6	Hide-and-Seek (2018)	68
3.6.7	Mozi (2019)	69
3.6.8	IoTroop / Reaper (2017)	69
3.6.9	More recent families (2022–2025)	69
3.7	Mirai Specifically – What to Know Cold	70
3.7.1	The infection lifecycle, end to end	70
3.7.2	The killer routine	71
3.7.3	Memory residence	71
3.7.4	The IP exclusion list	72
3.7.5	The DDoS attack types	72
3.7.6	Reading the Mirai source	72
3.8	Honeypots and Data Collection	73
3.8.1	What honeypots are	73

3.8.2	Cowrie	74
3.8.3	IoT POT	74
3.8.4	Honware (2019)	74
3.8.5	The bias problem in honeypot data	75
3.8.6	Where to find samples	75
3.9	Track 3 Self-Check Questions	76
4	Machine Learning for Security	77
4.1	The Mindset of ML in Security	77
4.1.1	Why security ML is different from textbook ML	77
4.1.2	The base rate problem	78
4.1.3	Precision, recall, F1, ROC, PR	78
4.1.4	Concept drift	79
4.1.5	Adversarial robustness	79
4.2	Feature Engineering for Syscall and Behavioral Data	80
4.2.1	Frequency vectors	80
4.2.2	N-grams	80
4.2.3	Sliding-window histograms	80
4.2.4	Sequence models on raw traces	81
4.2.5	Graph features	81
4.2.6	Cross-feature combinations	81
4.3	Classifiers	82
4.3.1	Random Forest	82
4.3.2	Gradient-boosted trees	82
4.3.3	SVMs	83
4.3.4	Neural networks for sequences	83
4.3.5	Anomaly detection	83
4.4	Evaluation Methodology	84
4.4.1	Temporal splits	84
4.4.2	Spatial separation	84
4.4.3	Class proportions	84
4.4.4	Hyperparameter tuning	85
4.4.5	Reporting standards	85
4.4.6	The TESSERACT framework	85
4.5	The Methodological Pathologies of Security ML	86
4.5.1	The Arp et al. 2022 paper: “Dos and Don’ts of Machine Learning in Computer Security”	86
4.5.2	What to do about it	87
4.5.3	Related: replicability and the security ML reproducibility crisis	87
4.6	The Cost-of-Collection Dimension	87
4.7	Bringing It Together	88
4.8	Track 4 Self-Check Questions	89

Introduction

These notes are a study guide for understanding behavioral detection of malware in IoT systems. The topic sits at the intersection of four fields: operating-system internals, embedded systems engineering, malware analysis, and machine learning. To do meaningful work in any one of them, you need working knowledge of the other three. This document tries to give you that working knowledge.

The notes are organized into four tracks:

1. **Linux internals & OS theory** – the substrate that all behavioral signals come from.
2. **IoT & embedded systems** – what’s distinctive about the target devices.
3. **Malware analysis as a discipline** – the field’s conceptual framework and the IoT malware landscape.
4. **Machine learning for security** – how behavioral data is actually classified, and the pitfalls that have plagued the literature for decades.

Each track is largely self-contained but the tracks build on each other in order. Concepts that appear in multiple tracks are explained where they first arise and cross-referenced later.

The intent is for these notes to give you most of the conceptual background you need, so that when you go to read papers, run experiments, or study source code, you’re filling in details rather than learning fundamentals. Boxes throughout the text highlight key ideas ([green](#)), warnings ([orange](#)), hands-on exercises ([purple](#)), deeper dives that can be skipped on a first read ([blue](#)), and pointers to further reading ([gray](#)).

1 Linux Internals & Operating System Theory

This track covers the substrate of behavioral analysis. Every signal you'll work with – syscall traces, process activity, network behavior, filesystem operations – is produced by the Linux kernel as it executes user programs. Without a clear mental model of what the kernel is doing, behavioral signals look arbitrary. With one, they become meaningful events you can reason about.

The investment here pays back across every other track. Reading firmware, understanding malware, designing detectors – all of these involve thinking about kernel-mediated behavior. If you only deeply learn one of the four tracks in this guide, learn this one.

1.1 The User/Kernel Boundary

1.1.1 Why the boundary exists

Modern CPUs implement *privilege levels*: hardware-enforced separations between trusted code (the kernel) and untrusted code (user programs). Intel calls them “rings”, ARM calls them “exception levels”, but the principle is universal. The Linux kernel runs at maximum privilege; user programs run at minimum privilege. A misbehaving user program – whether buggy or actively malicious – cannot corrupt the kernel or other processes because the hardware physically refuses to execute privileged instructions in user mode.

This separation is the foundation of every modern operating system. Without it, every program could crash the whole system; one program could read another's memory at will; a single bug in any application could compromise the entire machine. The kernel provides isolation, mediates shared resources (memory, devices, network), and exposes a controlled interface (syscalls) through which user programs can request services.

On x86-64, there are technically four rings (0–3) but Linux only uses two:

- **Ring 0** (kernel mode). The CPU can execute any instruction, access any memory, communicate with hardware directly, modify the page tables that isolate processes, and disable interrupts. The kernel runs here.
- **Ring 3** (user mode). The CPU refuses to execute privileged instructions like `cli/sti` (disable/enable interrupts), `lgdt` (load the global descriptor table), or `mov` to control registers. Attempting one raises a fault, which the kernel catches and typically converts to a SIGSEGV that kills the offending process. User code can only access memory the kernel has explicitly mapped for it, and only with the permissions (read, write, execute) the kernel set.

ARM (specifically AArch64, which dominates IoT) uses “exception levels” instead:

- **EL0** – user code (analogous to ring 3).
- **EL1** – the kernel (analogous to ring 0).
- **EL2** – hypervisor mode, used for virtualization.
- **EL3** – secure monitor / TrustZone firmware.

For our purposes, EL0 and EL1 are the equivalents of ring 3 and ring 0.

Key idea

The user/kernel boundary is enforced by hardware, not software. A bug in user code, or a maliciously crafted program, cannot bypass it – the CPU literally refuses to execute privileged operations in user mode. This is why the boundary is trustworthy. It's also why crossing the boundary is unavoidable when a user program needs the kernel's help: there's no other way for user code to interact with hardware, files, or other processes.

1.1.2 Crossing the boundary: the syscall mechanism

When a user program needs the kernel to do something – read from a file, send a packet, create a process – it must transition into kernel mode. This is called a *system call* or *syscall*. The mechanism is a special CPU instruction that traps to a kernel-mode entry point.

The exact instruction depends on the architecture:

- x86-64: the `syscall` instruction (introduced with AMD64; faster than the older mechanism).
- x86 (32-bit, legacy): `int 0x80` – a software interrupt.
- AArch64 (ARMv8): `svc #0` – “supervisor call”.
- 32-bit ARM (EABI): `swi 0x0` – “software interrupt”.
- MIPS: `syscall`.
- RISC-V: `ecall` – “environment call”.

In every case, the instruction conceptually does the same thing: it transitions the CPU to kernel mode and jumps to a pre-registered kernel entry point. The kernel then reads the syscall number and arguments from CPU registers (the exact registers depend on the architecture's *ABI* – application binary interface), dispatches to the appropriate handler, runs it, places the return value in a register, and executes a corresponding “return from exception” instruction (`sysret` on x86-64, `eret` on ARM) that switches back to user mode and resumes the calling program.

1.1.3 What happens during a syscall, step by step

Let's trace through a single syscall on x86-64 in detail. Suppose your program calls `write(1, "hi\n", 3)` – write the bytes "hi\n" to file descriptor 1 (stdout).

User-mode preparation. The C library's `write()` wrapper function loads:

- `rax` ← 1 (the syscall number for `write` on x86-64 Linux)
- `rdi` ← 1 (first argument: the file descriptor)
- `rsi` ← pointer to "hi\n" (second argument: the buffer)
- `rdx` ← 3 (third argument: the length)

The syscall instruction. The wrapper executes the `syscall` instruction. At the hardware level, this causes the CPU to:

1. Save the current instruction pointer (`rip`) into `rcx`.
2. Save the current flags register (`rflags`) into `r11`.
3. Mask the interrupt flag in `rflags` according to a saved mask.

4. Load `rip` from the `LSTAR` model-specific register (MSR), which was set up at boot to point to `entry_SYSCALL_64` in the kernel.
5. Switch to ring 0 (kernel mode).

Notice the side effect: `rcx` and `r11` are clobbered. This is why the syscall calling convention differs from the regular function calling convention. Normal x86-64 functions pass their fourth argument in `rcx`; syscalls pass it in `r10` instead, because `rcx` is unusable.

Kernel entry. Execution is now at `entry_SYSCALL_64`. This function:

1. Swaps the GS segment to point to per-CPU kernel data.
2. Saves the user-space registers on the kernel stack (so they can be restored later).
3. Looks up the kernel stack pointer for the current task and switches `rsp` to it.
4. Validates that the syscall number (in `rax`) is within bounds.
5. Looks up `sys_call_table[1]` – the function pointer for `write`.
6. Calls that function with the arguments from `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`.

The handler runs. The kernel's `sys_write` function:

1. Looks up file descriptor 1 in the current process's file descriptor table.
2. Verifies the user-supplied buffer pointer is in user memory (not kernel memory) and is readable.
3. Calls the underlying “`vfs_write`” function, which dispatches based on what kind of file descriptor it is (a file on disk, a pipe, a socket, etc.).
4. For stdout pointing to a terminal: the data is queued for the tty subsystem.
5. Returns the number of bytes written (or a negative error code).

Returning to user mode. The kernel:

1. Places the return value in `rax`.
2. Restores user-space registers from the kernel stack.
3. Switches the GS segment back to user.
4. Executes `sysretq`, which restores `rip` from `rcx` and `rflags` from `r11`, switching back to ring 3.

The C wrapper resumes. The C library's `write()` checks `rax`: if negative, set `errno` and return -1; otherwise return the value. Control returns to your program, which sees a successful `write()` call.

This is one syscall. It happens billions of times a day on every Linux system.

1.1.4 Why crossing the boundary is expensive

A syscall is much slower than a regular function call. A normal function call on a modern CPU takes about 1 ns (a couple of clock cycles for `call` and `ret`, prediction handles the rest). A syscall takes 100–1000 ns even for a no-op syscall like `getpid()`. The reasons:

- **Mode switch overhead.** The CPU has to flush its pipeline, change privilege level, swap stacks, save and restore registers. This is fundamentally serial work.

- **Cache effects.** Kernel code paths may not be in the L1 instruction cache, especially for less-common syscalls. The kernel's data structures (per-CPU variables, task structures) need to be loaded into cache.
- **TLB (Translation Lookaside Buffer) flushes.** The TLB caches virtual-to-physical address translations. After Meltdown (2018), Linux added Kernel Page Table Isolation (KPTI) which gives the kernel a separate page table; every syscall now involves a TLB flush, adding another 100+ cycles.
- **Spectre / Meltdown mitigations.** Each syscall now executes additional mitigation code that wasn't there before 2018. The exact cost varies by CPU.
- **Saving / restoring state.** On Intel CPUs with AVX, the kernel may need to save extended FPU state. This is amortized but still adds up.

The total varies enormously by workload, but a rough rule of thumb: a program that makes one syscall per memory access (every nanosecond) would slow down by 100–1000x. A program that makes one syscall per millisecond would slow down by maybe 0.1%.

Going deeper

This expense is the entire reason eBPF exists in the form it does. Imagine you want to monitor every syscall in the system for security analysis. A naive approach: send each syscall event to a userspace daemon for analysis. But *sending* requires the daemon to be scheduled, which is itself a syscall worth of overhead. You've just doubled the syscall cost. For a busy server doing 100,000 syscalls/second, that means burning a full CPU core on overhead.

eBPF solves this by running the analysis logic *inside the kernel*, at the syscall entry point itself. No context switch, no scheduling, no data movement. The kernel decides in-place whether to log the event, and only events that pass an in-kernel filter ever leave the kernel. This makes monitoring practical at near-zero overhead, which is why every modern security monitoring tool is built on eBPF.

1.1.5 The vDSO: not all "syscalls" are syscalls

Some operations are called extremely often but don't fundamentally require kernel privileges. The canonical example is `gettimeofday()`: reading the current time. Conceptually you're asking the kernel "what time is it", but in practice the kernel maintains a free-running counter that user processes could safely read directly if you gave them access.

Linux provides exactly this via the *vDSO* (virtual Dynamic Shared Object). The vDSO is a small shared library that the kernel maps into every process's address space. When you call `gettimeofday()`, libc actually jumps into the vDSO, which reads kernel-maintained timing data from a shared page *without* crossing the privilege boundary. No mode switch, no overhead beyond a function call.

Other vDSO entries on most systems: `clock_gettime`, `getcpu`, `time`. The exact set depends on the architecture.

Watch out

For behavioral analysis, this means: **strace** and other ptrace-based tools *don't see* vDSO calls, because no syscall actually happens. The process is asking the kernel for time, but the kernel never gets involved. If you're building a behavioral profile and you wonder why a program "does nothing" for a while, it may be making millions of vDSO calls invisible

to your tracer. eBPF can attach to user-space functions via uprobes if you really need to see them, but at significant overhead.

Try this

Run `cat /proc/self/maps | grep vdso` – you’ll see the vDSO mapped into your shell’s address space. Then `ldd /bin/ls` – you’ll see “linux-vdso.so.1” as one of the “libraries” that `ls` “links” against. It’s a real binary object you can dump and analyze, even though it lives only in memory.

Further reading

- [Linux Inside \(0xAX\)](#), “System Calls” chapter – detailed kernel-side walkthrough.
- `man 2 syscall` – canonical reference, includes the per-architecture register table.
- [LWN](#), “Anatomy of a system call, part 1” by David Drysdale.
- Kerrisk, *The Linux Programming Interface* (TLPI), Chapter 3 §3.1.
- LiveOverflow, “Syscalls, Kernel vs. User Mode and Linux Kernel Source Code” (bin 0x09) on YouTube.
- [Linux kernel docs: x86_64 entry point](#) – the actual code.

1.2 The Syscall Interface

1.2.1 Syscall numbers

Every syscall is identified by a small integer. The kernel has a table (`sys_call_table`) indexed by this number, mapping each to its handler function. On x86-64 Linux, `read` is 0, `write` is 1, `open` is 2, `close` is 3, and so on. The full list is in `/usr/include/asm/unistd_64.h` on most systems, or via `ausyscall -dump` if you have audit tools installed. Current Linux has somewhere around 350–450 syscalls depending on architecture.

Key idea

Syscall numbers are architecture-specific. `read` is #0 on x86-64, #3 on 32-bit ARM (EABI), #63 on AArch64, and #4003 (a Linux-specific offset above 4000) on MIPS o32. The *names* are stable; the *numbers* are not.

This is enormously consequential for any system that processes syscall traces across architectures. If your detector trains on x86-64 traces and sees the number 0, it learned to associate `read` with whatever pattern that involved. Run it on ARM traces and 0 means `restart_syscall`, a totally different operation. The traces become noise.

Two correct approaches:

1. Normalize numbers to names before feature extraction. Lookup tables exist for every architecture.
2. Train separate models per architecture, accepting the cost.

A wrong approach is to silently use raw syscall numbers as features in a cross-architecture detector. This kind of subtle bug has appeared in published work.

1.2.2 The syscall ABI by architecture

How are arguments passed to syscalls? The convention – which registers hold which arguments – is defined by each architecture’s *kernel ABI*. The kernel ABI often differs slightly from the user-space *function call ABI* for the same architecture, because the syscall instruction itself clobbers some registers.

Architecture	Syscall # in	Argument registers (in order)
x86-64	rax	rdi, rsi, rdx, r10, r8, r9
i386 (32-bit)	eax	ebx, ecx, edx, esi, edi, ebp
ARM/EABI (32-bit)	r7	r0, r1, r2, r3, r4, r5
AArch64 (ARM64)	w8/x8	x0, x1, x2, x3, x4, x5
MIPS (o32)	v0	a0, a1, a2, a3, stack...
RISC-V	a7	a0, a1, a2, a3, a4, a5
PowerPC	r0	r3, r4, r5, r6, r7, r8

Return values follow a different convention:

- x86-64 returns in **rax**.
- AArch64 returns in **x0**.
- MIPS returns in **v0**, with the error flag in **a3**.
- Negative return values typically indicate errors (with the magnitude being the **errno** value), but the exact convention varies. MIPS uses a separate error flag instead of negative returns. Some architectures (Alpha, IA-64) have a second return-value register for syscalls that need to return two values (like **pipe()**).

1.2.3 The syscalls you'll see constantly

Out of the 350 syscalls in the kernel, a few dozen account for the vast majority of activity. Understanding what each does is essential for reading syscall traces:

File I/O. `read`, `write`, `open`, `openat`, `close`, `stat`, `fstat`, `lstat`, `newfstatat`, `lseek`, `pread64`, `pwrite64`, `readv`, `writev`.

Most modern Linux programs use **openat** rather than **open**. **openat** takes an additional file descriptor argument that specifies a directory to resolve relative paths against; this prevents race conditions where the meaning of a path changes between resolution and use. The C library has migrated to **openat** internally, so you'll see it dominate file-open traffic.

Memory management. `mmap`, `mprotect`, `munmap`, `brk`, `madvise`, `mlock`, `munlock`.

brk extends the data segment (used for small heap allocations); **mmap** creates new memory mappings (used for large allocations, files, and shared libraries). **mprotect** changes permissions on existing mappings – a particularly important syscall for security analysis because it's how injected code becomes executable.

Process management. `fork`, `vfork`, `clone`, `execve`, `execveat`, `exit`, `exit_group`, `wait4`, `waitid`, `kill`, `tgkill`, `getpid`, `getppid`, `getuid`, `geteuid`, `setuid`, `seteuid`, `prctl`.

Networking. `socket`, `bind`, `listen`, `accept`, `accept4`, `connect`, `sendto`, `recvfrom`, `sendmsg`, `recvmsg`, `getsockopt`, `setsockopt`, `shutdown`, `getsockname`, `getpeername`.

IPC and event handling. `pipe`, `pipe2`, `dup`, `dup2`, `dup3`, `epoll_create1`, `epoll_ctl`, `epoll_wait`, `epoll_pwait`, `poll`, `select`, `eventfd`, `signalfd`, `timerfd_create`.

Signals. `rt_sigaction`, `rt_sigprocmask`, `rt_sigreturn`, `kill`, `tgkill`.

Control. `ioctl`, `fcntl`, `ptrace`, `seccomp`.

Filesystem manipulation. `mkdir`, `rmdir`, `rename`, `link`, `unlink`, `symlink`, `chmod`, `chown`, `access`, `faccessat`.

Specific to behavioral analysis. A few syscalls are particularly informative for malware detection:

- `execve` – a new program is being run. Always interesting.
- `ptrace` – often used by malware as an anti-debugging check (called on self to see if a debugger is already attached).
- `prctl(PR_SET_NAME, ...)` – changes the process’s visible name in `ps` output. Sometimes used to hide.
- `mprotect(..., PROT_EXEC)` on previously non-executable pages – often signals code injection or self-modifying code.
- `setsid()` – detaches from controlling terminal, the classical daemonization signal.
- `kill` on many different target PIDs – consistent with a “killer” routine that’s targeting other processes.
- `socket` followed by many `connects` in quick succession – a port scanner or brute-force connection attempt.

1.2.4 Libc wrappers vs raw syscalls

When a C program calls `fopen("/etc/passwd", "r")`, that’s not a syscall. `fopen` is a libc function. Internally it calls `open()` (a libc function), which is a thin wrapper around the actual `openat` syscall. The wrapper:

1. Loads the syscall number into the appropriate register.
2. Loads arguments into the appropriate registers.
3. Executes the syscall instruction.
4. On return, checks the result. If negative, sets `errno` to the negated value and returns -1; otherwise returns the value as-is.

Some libc functions don’t correspond to a single syscall:

- `malloc` – uses `brk` for small allocations, `mmap` for large ones, internal heap management for the rest. A single `malloc(8)` call may make zero syscalls (using a previously-extended heap).
- `printf` – formats the string, then calls `write`. The formatting is libc; the I/O is the syscall.
- `fopen` – calls `openat` and sets up libc’s buffering machinery.
- `getpwnam` – may read `/etc/passwd` (multiple syscalls), consult NSS modules (more syscalls), even contact LDAP servers (network syscalls), depending on system configuration.

Tracing tools work at different levels:

- `strace` traces syscalls (the kernel boundary).
- `ltrace` traces library calls (the libc/program boundary). Less reliable; doesn’t work on all binaries.

- **perf trace** traces syscalls with lower overhead than **strace**, using ftrace/eBPF.
- eBPF tools can trace either, depending on where you attach.

Try this

Run **strace -c ls /tmp** and look at the syscall histogram. You'll see something like 60–80 distinct syscalls, dominated by **newfstatat**, **getdents64**, **close**, **openat**. Now **strace -c curl https://example.com -o /dev/null** – very different distribution. Heavy on networking syscalls: **socket**, **connect**, **poll**, **recvfrom**, **sendto**. Then **strace -c python3 -c "print(1)"** – a huge number of **openat** and **newfstatat** from Python loading its module hierarchy at startup. The actual “print(1)” is a tiny fraction. The point: every program has a recognizable syscall *shape*, and behavioral classifiers learn these shapes.

Further reading

- **man 2 syscalls** – the canonical list.
- **man 2 syscall** – the ABI table per architecture.
- [Searchable syscall table by architecture](#) – compare x86, ARM, MIPS side by side.
- Kerrisk, *TLPI*, Chapter 3 §3.2–3.4.
- [Filippo's Linux syscall table for x86-64](#).

1.3 The Process Model

1.3.1 Tasks, processes, threads

The Linux kernel's unit of scheduling is the *task*, represented internally by a **task_struct** data structure. A task is a single thread of execution. What we informally call a “process” or a “thread” are both tasks in the kernel's view – they differ only in what they share with other tasks.

When you call **fork()**, you create a new task with its own private address space, file descriptor table, signal handlers, etc. – a process.

When you call **pthread_create()**, you create a new task that shares the address space, file descriptors, and signal handlers with the calling task – a thread.

Both are implemented via the same underlying syscall: **clone()**. **clone** takes a flags argument that specifies what to share. Different flag combinations produce different sharing patterns, and the various “higher-level” creation primitives (**fork**, **vfork**, threads, containers) are all specific **clone** flag combinations.

Key idea

The kernel's view: every task is a task. Processes vs threads is a userspace abstraction layered on top via different **clone** flags. This is what people mean when they say “Linux doesn't really distinguish processes and threads”.

1.3.2 PIDs and TIDs

Linux's terminology is unfortunately confusing here. The kernel uses:

- **Thread ID (TID, sometimes “PID” in kernel docs)** – the unique identifier of a task. Every task has one.
- **Thread Group ID (TGID, “PID” in userspace)** – shared by all tasks in the same “process”. The main thread’s TID equals its process’s TGID.

Userspace tools (`ps`, `top`, `kill`) typically show TGIDs and call them “PIDs”. The system call `getpid()` returns the TGID; `gettid()` returns the actual TID.

For a single-threaded process: TID = TGID. For a process with N threads, you’ll have N distinct TIDs all sharing one TGID.

If you look at `/proc`:

- `/proc/<TGID>/` – the process directory; everything visible here is process-wide.
- `/proc/<TGID>/task/<TID>/` – per-thread information within that process.

1.3.3 fork, vfork, clone, exec

These four are often confused. Here’s what each actually does:

fork(). Creates a new process that is a near-perfect copy of the calling process. The child:

- Has a copy of the parent’s entire address space (implemented as copy-on-write: pages are shared until one process modifies them).
- Inherits all open file descriptors (but they’re shared, not copied – if the parent reads from FD 3, the offset advances for both).
- Inherits signal handlers, environment variables, current directory, resource limits.
- Has a different PID, and its parent PID points to the original process.

`fork` returns 0 in the child and the child’s PID in the parent, enabling the classic pattern:

```
pid_t pid = fork();
if (pid == 0) {
    // child code
} else if (pid > 0) {
    // parent code; pid is the child's PID
} else {
    // fork failed
}
```

vfork(). A historical optimization. Like `fork` but the parent is suspended until the child calls `execve()` or `exit()`, and the child temporarily shares the parent’s memory (not copy-on-write – actually shared). Was important for performance before copy-on-write made `fork` cheap. Mostly obsolete; still used in some embedded code.

clone(). The universal primitive. Takes a flags argument controlling what’s shared:

- `CLONE_VM` – share address space (makes it a thread, not a process).
- `CLONE_FS` – share filesystem state (umask, current directory, root directory).
- `CLONE_FILES` – share the file descriptor table.
- `CLONE_SIGHAND` – share signal handlers.

- `CLONE_THREAD` – new task is in the same thread group.
- `CLONE_NEWPID`, `CLONE_NEWNET`, `CLONE_NEWNS`, ... – create new namespaces (the foundation of containers).
- `CLONE_PTRACE` – if the parent is being traced, trace the child too.
- `CLONE_PARENT_SETTID`, `CLONE_CHILD_SETTID`, `CLONE_CHILD_CLEARTID` – thread ID handling.

The standard pthread library calls something like:

```
clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND |
      CLONE_THREAD | CLONE_SYSVSEM | CLONE_SETTID |
      CLONE_PARENT_SETTID | CLONE_CHILD_CLEARTID, ...)
```

`fork` is essentially `clone(SIGCHLD, NULL)` – no sharing flags, just create an independent task.

execve(). Replaces the current process’s program. The PID stays the same; everything else (address space, code, data) is wiped and replaced with the new program loaded from disk. After `execve`, your process is running a different program with the same PID and inherited file descriptors (unless they had close-on-exec set).

This is why the typical shell command flow is fork-then-exec: the shell forks a child, which immediately execs the desired program. The shell’s parent stays alive to wait for the child.

Key idea

The separation of fork and exec is one of Unix’s elegant design decisions. It lets a parent set up the child’s environment between the fork and the exec – redirect file descriptors (the foundation of shell pipes and redirections), drop privileges, change directory, set resource limits – without complicated argument-passing to the new program. Windows’s `CreateProcess` bundles all of this into one call, which is simpler in some ways but loses this composability.

1.3.4 Process states

Every task is always in one of a small number of states. You can see the current state in `/proc/<pid>/status` (the “State” line) or via `ps -o stat:`

- **R (Running)** – currently executing on a CPU, or sitting in the run queue ready to be scheduled. At any moment, only as many tasks can be “running” (on a CPU) as you have CPU cores; the rest are “runnable”.
- **S (Interruptible sleep)** – waiting for some event (data on a socket, a timer to expire, a signal). Can be woken by signals. Most tasks are in S state at any given moment – they’re just waiting for something to happen.
- **D (Uninterruptible sleep)** – waiting for an I/O operation that cannot be interrupted (typically disk I/O at the block-device level). Cannot be killed even with `SIGKILL` until the I/O completes. If a process is stuck in D state for a long time, it usually means hardware trouble.
- **T (Stopped)** – explicitly stopped, either by a signal (`SIGSTOP`, `SIGTSTP`) or by a debugger (`ptrace`).

- **t (Tracing stop)** – stopped specifically because it's being traced. Distinguished from regular T on modern kernels.
- **Z (Zombie)** – has exited, but the parent hasn't called `wait()` yet. The task structure persists just to hold the exit status until reaped.
- **X (Dead)** – truly gone. You rarely see this; the kernel removes the structure almost immediately.

1.3.5 Process groups, sessions, and controlling terminals

These exist primarily for terminal management and job control in shells:

A **process group** is a set of processes that can be signaled together. When you type `ls | grep foo | wc -l` in a shell, the shell creates these three processes as members of one process group. Pressing Ctrl-C sends SIGINT to the entire group at once, so the whole pipeline dies.

A **session** is a set of process groups, typically all the processes spawned by one shell or login. The shell is the session leader. A session may have a **controlling terminal** – the tty/pty associated with it.

The syscall `setsid()` creates a new session with the calling process as the session leader, detaching from any controlling terminal. This is exactly what a daemon does at startup: `fork`, parent exits, child calls `setsid`. The child is now in its own session with no controlling terminal – it can't be killed by Ctrl-C, it doesn't get SIGHUP when the shell closes, it survives independently.

Key idea

Behavioral relevance: a process that calls `setsid()` and `fork()` (often called the “double-fork” pattern) is daemonizing. Many malware samples do this. A detection rule like “new process that calls `setsid` early in its lifetime and persists” catches this.

1.3.6 Signals

Signals are asynchronous notifications delivered to processes. There are about 30 standard signals plus a range of real-time signals (SIGRTMIN through SIGRTMAX).

Common signals:

- **SIGTERM (15)** – polite termination request. The default of `kill`. Process can catch it and clean up.
- **SIGINT (2)** – terminal interrupt (Ctrl-C). Catchable.
- **SIGKILL (9)** – forceful termination. *Cannot be caught or ignored*. The kernel always honors it.
- **SIGSTOP (19)** – stop the process. *Cannot be caught or ignored*. Pair with SIGCONT to resume.
- **SIGCONT (18)** – continue a stopped process.
- **SIGSEGV (11)** – segmentation fault (invalid memory access). Default action: dump core and exit.
- **SIGCHLD (17)** – child process state change (typically: child has exited).
- **SIGUSR1 (10), SIGUSR2 (12)** – user-defined.

- **SIGPIPE (13)** – writing to a pipe with no readers.
- **SIGHUP (1)** – hangup; sent to processes when their controlling terminal closes.

Key facts about signal handling:

- Most signals can be caught (handled by user code) or ignored. The two exceptions, SIGKILL and SIGSTOP, are unconditional kernel actions.
- Signal delivery happens at well-defined points: when the kernel is about to return to user mode. So a process that's stuck in a syscall in uninterruptible D state won't see signals until it returns – which is why D-state processes can't be killed.
- Signals are coalesced by default. If SIGUSR1 arrives twice while masked or pending, the process sees it once. Real-time signals (SIGRTMIN through SIGRTMAX) are queued, but the standard signals aren't.
- A process can mask signals temporarily with `sigprocmask`, deferring delivery until later.

1.3.7 The /proc filesystem

`/proc` is a virtual filesystem – there are no actual files on disk; the kernel generates the contents on demand when you read. It exposes kernel data structures as readable text (mostly). For every running process there's a directory `/proc/<pid>/` containing:

- **status** – human-readable summary: name, state, PIDs, UIDs/GIDs, memory usage, signal masks.
- **stat** – one-line machine-readable status with dozens of fields. Used by `ps`, `top`.
- **cmdline** – the command-line arguments the process was started with (null-separated).
- **comm** – the process name (max 16 characters), modifiable by `prctl(PR_SET_NAME)`.
- **exe** – a symlink to the actual executable file.
- **cwd** – a symlink to the current working directory.
- **root** – a symlink to the root directory (different for chrooted processes).
- **maps** – the process's virtual memory mapping (which addresses contain what).
- **smaps** – like maps but with detailed memory usage statistics.
- **fd/** – a directory of symlinks, one per open file descriptor.
- **fdinfo/** – per-FD detailed information (offset, flags).
- **environ** – environment variables (null-separated).
- **ns/** – the process's namespaces (used for containers).
- **task/** – subdirectories per thread (TID) within this process.

There are also system-wide files: `/proc/cpuinfo` (CPU details), `/proc/meminfo` (memory statistics), `/proc/net/tcp` (TCP connection table), `/proc/modules` (loaded kernel modules), `/proc/version`, and many more.

Watch out

Many malware samples and analysis evasion routines read `/proc` files for environment fingerprinting. Common checks:

- `/proc/cpuinfo` – check CPU model for VM signatures or unusually few cores.
- `/proc/scsi/scsi`, `/proc/ide/` – check for VM-specific device strings.
- `/proc/self/status` – check the `TracerPid` field (non-zero if being traced).
- `/proc/sys/kernel/random/uuid` – generate something that should be unique per machine.
- `/proc/<pid>/maps` for every process – enumerate what’s running, looking for analysis tools.

A process opening these files in unusual combinations early in its execution is doing environment fingerprinting – a strong signal that it expects to be analyzed.

Try this

Several useful explorations:

- `ls -la /proc/self/fd/` – see your shell’s open file descriptors as symlinks. FDs 0, 1, 2 point to your terminal.
- `cat /proc/self/maps` – see the address-space layout of `cat`: code, libraries, stack, vDSO.
- `cat /proc/self/status | head -20` – the human-readable status of your shell.
- `cat /proc/cpuinfo | head -25` – CPU info as the kernel sees it.
- For a longer-running process: `ls -la /proc/$(pidof firefox)/fd/ | head -20` – see all the files and sockets a real application has open.

Further reading

- Kerrisk, *TLPI*, Ch. 6 (Processes), 24–28 (creation and termination), 20–22 (signals).
- Robert Love, *Linux Kernel Development*, Ch. 3 (Process Management), 5 (System Calls).
- `man 5 proc` – canonical reference; long but readable.
- `man 2 clone` – essential for thread internals.

1.4 Memory Model

1.4.1 Virtual memory: the abstraction

Every process on a modern operating system sees its own private 64-bit (on 64-bit machines) virtual address space, even though physical RAM is much smaller and shared between all processes. This illusion is implemented by the kernel and CPU jointly:

- The kernel maintains, for each process, a set of *page tables* that map virtual addresses to physical addresses. Each entry covers a 4 KB page (or larger “huge pages” for performance).
- The CPU’s *Memory Management Unit (MMU)* consults the page tables on every memory access to translate the virtual address to a physical one. This happens in hardware, transparently to the program.
- Recently-used translations are cached in the *Translation Lookaside Buffer (TLB)* for speed – a fully-fledged page-table walk would take dozens of memory accesses by itself.
- When a program accesses a virtual address that’s not currently mapped to physical memory, the MMU raises a *page fault*, which the kernel handles. The kernel may load the page from disk (demand paging), allocate a new physical page, or kill the process for accessing invalid memory.

Why this matters: virtual memory is what gives processes isolation, lets the kernel implement copy-on-write for `fork`, makes `mmap`-ing files possible, and enables ASLR (which randomizes where things land in the address space).

1.4.2 The address space layout of a process

On x86-64 with 48-bit virtual addresses (the common case), a process's virtual address space is split:

- `0x0000_0000_0000_0000` to `0x0000_7fff_ffff_ffff` – user space (the lower 128 TB).
- `0x0000_8000_0000_0000` to `0xffff_7fff_ffff_ffff` – non-canonical hole. Hardware refuses to access these; any attempt causes a fault.
- `0xffff_8000_0000_0000` to `0xffff_ffff_ffff_ffff` – kernel space (upper 128 TB). User mode cannot access; the kernel maps itself here.

The non-canonical hole exists because the architecture uses “canonical addresses” where the top 16 bits must all be 0 or all be 1; addresses in between are invalid. This effectively reserves room for future architectural expansion to a full 64-bit address space.

Within the user space portion, a typical process is laid out roughly like this (from low to high addresses):

1. **Code (.text section).** Read+execute, not writable. Contains the compiled machine instructions. For non-PIE binaries this is at a fixed address (traditionally around `0x400000` on x86-64). For position-independent executables (modern default), it's at a randomized address.
2. **Read-only data (.rodata).** Read-only. Contains string literals, const arrays.
3. **Initialized data (.data).** Read+write. Contains initialized global variables.
4. **Uninitialized data (.bss).** Read+write, zero-initialized. Contains uninitialized global variables; takes no space in the binary on disk.
5. **Heap.** Grows upward. Managed by `malloc`, which extends it via `brk()` for small allocations or `mmap()` for large ones.
6. **mmap region.** Shared libraries, file mappings, large memory allocations from `malloc`, thread stacks. Lives in the middle of the address space, typically growing downward.
7. **Stack.** Grows downward from a high address. Contains function call frames, local variables.
8. **vDSO and vvar.** Small kernel-shared pages at high addresses.

You can see this for any process by reading `/proc/<pid>/maps`. Each line shows a mapped range and its permissions.

1.4.3 ASLR

Address Space Layout Randomization, enabled by default in modern Linux, randomizes:

- The starting address of the stack.
- The starting address of the heap (more precisely, the `brk` break).
- The starting address of the `mmap` region.
- For PIE (Position-Independent Executable) binaries, the binary itself.

The randomization gives each process a unique layout, frustrating exploits that depend on hardcoded addresses. The amount of randomization varies: on 64-bit Linux, the mmap region can be placed in any of 2^{30} positions, the stack in fewer (because it must be aligned and have room to grow).

Watch out

ASLR is often disabled or partially disabled on IoT devices for performance/simplicity reasons. This makes those devices easier to exploit – a working exploit doesn't need to defeat ASLR. When you're analyzing IoT firmware in an emulator, this is one of the factors that makes the threat model different from desktop Linux.

1.4.4 mmap and mprotect

`mmap` creates a new memory mapping. Two main uses:

File mappings. Map a file into memory. Reads and writes to the mapped memory go to the file. Used for shared libraries (`ld.so` mmaps every `.so` file), memory-mapped I/O, and shared memory between processes.

Anonymous mappings. Allocate fresh memory not backed by a file. Used by `malloc` for large allocations, by `pthread_create` for thread stacks, and any time a program needs a chunk of memory at a specific size or alignment.

The `mmap` syscall takes:

- A length in bytes.
- Protection flags: `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`.
- Flags: `MAP_SHARED` or `MAP_PRIVATE`, `MAP_ANONYMOUS`, etc.
- A file descriptor (or -1 for anonymous).
- An offset.

`mprotect` changes the protection on an existing mapping. You can flip a page from writable to read-only, or from non-executable to executable.

Key idea

The `mprotect-to-exec` pattern is a high-signal indicator of code injection. The classical attack:

1. Allocate memory: `mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_ANONYMOUS | MAP_PRIVATE, -1, 0)`
2. Write shellcode into the allocation.
3. Make it executable: `mprotect(addr, size, PROT_READ | PROT_EXEC)`
4. Jump to it.

This is also done by legitimate JIT compilers (Java, Node.js, Python with various JITs). But for a process that has no business being a JIT – a router daemon, an IoT camera service – this pattern is highly suspicious. Behavioral detectors should track `mprotect` calls that add `PROT_EXEC` to previously non-executable pages.

On modern systems with W^X (write XOR execute) protections, this pattern is sometimes blocked outright at the kernel level. SELinux and AppArmor profiles can prohibit it for specific binaries.

1.4.5 ELF binary format

Linux executables are ELF (Executable and Linkable Format) files. ELF has two views:

The section view (for the linker, when you're *building* an executable). Sections are named regions of bytes with associated attributes. Common section names:

- `.text` – executable code.
- `.data` – initialized read-write data.
- `.rodata` – read-only data (string literals, const arrays).
- `.bss` – uninitialized data, zero-initialized at load. Takes no bytes in the file (just a size declaration).
- `.plt` (Procedure Linkage Table) – stubs for calling dynamically-linked functions.
- `.got` (Global Offset Table) – pointers patched up at runtime by the dynamic linker.
- `.symtab`, `.strtab` – symbol table and associated strings (for the linker).
- `.dynsym`, `.dynstr` – dynamic symbol table (for the dynamic linker).
- `.rela.dyn`, `.rela.plt` – relocation entries.
- `.init`, `.fini` – constructor and destructor code.
- `.note.*` – metadata sections (build ID, ABI tags, properties).
- `.debug_*` – DWARF debug information, usually stripped in production.

The segment view (for the loader, when the OS *runs* the binary). Segments are contiguous regions of bytes that get mapped into memory. The program header table describes each segment with a type, file offset, virtual address, file size, memory size, and permissions. The kernel doesn't care about sections at load time; it reads the program header table and mmap's each segment.

Important segment types:

- **LOAD** – a region to load into memory. Typically two: one for read-execute (code), one for read-write (data).
- **DYNAMIC** – contains info for the dynamic linker.
- **INTERP** – path to the dynamic linker itself (e.g., `/lib64/ld-linux-x86-64.so.2`).
- **NOTE** – metadata.
- **GNU_RELRO** – read-only after relocation.

For analysis:

```
file binary                # quick identification
readelf -h binary          # ELF header (architecture, type, entry
    point)
readelf -l binary          # program headers (segment view)
readelf -S binary          # section headers (section view)
readelf -d binary          # dynamic section (library dependencies)
readelf --dyn-syms binary  # imported and exported symbols
nm -D binary               # alternative for dynamic symbols
objdump -d binary          # disassembly
strings -a binary          # all printable strings
```

1.4.6 Dynamic linking and ld.so

When you run a dynamically-linked binary, the kernel doesn't directly execute your program. It executes the *dynamic linker* (typically `/lib64/ld-linux-x86-64.so.2` on x86-64), passing the path to your program as an argument.

The dynamic linker:

1. Reads your binary's program header, finds the INTERP and DYNAMIC segments.
2. Reads the DT_NEEDED entries from the dynamic section: a list of shared libraries needed (e.g., `libc.so.6`, `libpthread.so.0`).
3. For each library, searches a sequence of paths:
 - `LD_LIBRARY_PATH` environment variable (if set).
 - `/etc/ld.so.cache` (a cache built from `/etc/ld.so.conf` and standard system directories).
 - Default paths: `/lib`, `/lib64`, `/usr/lib`, `/usr/lib64`.
4. Maps each library into memory with `mmap`.
5. Resolves symbols. By default, lazy: the first call to a function triggers resolution via the PLT/GOT mechanism. Eager resolution is possible with `LD_BIND_NOW=1`.
6. Runs initializers (constructor functions, `_init` sections).
7. Jumps to your program's entry point (typically `_start` in the `libc` startup code, which eventually calls your `main()`).

Try this

Run `strace ls 2>&1 | head -50`. You'll see dozens of syscalls before `ls` produces any output. Most are the dynamic linker: `openat` on libraries, `mmap` to load them, `mprotect` to set their final permissions.

Specifically try `strace -e trace=openat ls` – a clean view of just the file opens. You'll see `/etc/ld.so.cache`, `/lib/x86_64-linux-gnu/libc.so.6`, and a few others, then the actual files `ls` cares about.

This is important context for behavioral analysis: every dynamically-linked binary's syscall trace starts with a substantial common prefix of loader activity, mostly identical across all binaries on a given system. Naive classifiers might learn this prefix as “normal behavior” and miss the actual program's behavior.

Going deeper

The PLT/GOT mechanism is worth understanding for security work. When your program calls a function from a shared library, the call doesn't go directly to the function (whose address isn't known until runtime). Instead:

1. The call goes to a stub in the PLT (Procedure Linkage Table).
2. On the first call, the stub jumps through the GOT (Global Offset Table) to a resolver function in `ld.so`.
3. The resolver finds the real address of the function and writes it into the GOT entry.

4. The resolver jumps to the real function.
5. On all subsequent calls, the PLT stub jumps through the GOT, which now contains the real address, going directly to the function.

The GOT is a normal data section that, after resolution, contains code-pointer values. This makes it an attractive target for exploits: corrupt a GOT entry, and you redirect every future call to that function.

Modern hardenings include “Full RELRO” (GOT is read-only after dynamic linking completes) and FORTIFY_SOURCE. Many IoT binaries lack these mitigations.

Further reading

- [“Anatomy of a Program in Memory” by Gustavo Duarte](#) – the clearest visual treatment of process address space.
- Kerrisk, *TLPI*, Ch. 49 (Memory Mappings), 50 (Virtual Memory Operations).
- Eli Bendersky’s blog on ELF: [Position-independent code](#) and [related ELF posts](#).
- `man 5 elf` – dense, but the canonical reference.
- LiveOverflow “Binary Exploitation” series episodes 1–5 on YouTube – visceral understanding of process memory.

1.5 File Descriptors and “Everything is a File”

1.5.1 What a file descriptor actually is

A file descriptor (FD) is a small non-negative integer that indexes into a per-process table maintained by the kernel. Each entry in this table refers to an *open file description* – a kernel data structure representing an open instance of some I/O resource.

The resource can be:

- A regular file on disk.
- A directory.
- A character device (terminal, `/dev/null`, `/dev/urandom`).
- A block device (a disk).
- A pipe (anonymous or named/FIFO).
- A socket (TCP, UDP, Unix domain).
- An event source: `epoll`, `eventfd`, `signalfd`, `timerfd`, `inotify`, `fanotify`.
- A memory file descriptor (`memfd_create`).
- A user-space filesystem (FUSE).
- Specialized handles like `perf_event_open` results, BPF maps, BPF programs.

When a syscall like `read(fd, buf, len)` is invoked, the kernel:

1. Looks up `fd` in the calling process’s file descriptor table.
2. Finds the open file description it points to.
3. Determines what kind of resource it is.
4. Dispatches to the appropriate handler (`ext4_file_read` for an ext4 file, `tcp_recvmmsg` for a TCP socket, etc.).

This is what people mean by “everything is a file” on Unix: many kinds of resources are accessed through the same uniform interface. The same `read` and `write` syscalls work on files, sockets, pipes, devices, terminals.

1.5.2 Standard file descriptors

Every process starts with three FDs open by convention:

- FD 0 – standard input (`stdin`).
- FD 1 – standard output (`stdout`).
- FD 2 – standard error (`stderr`).

These conventions are pure convention; the kernel doesn’t enforce them. A program can close any of them, or open new descriptors with any numbers it chooses (up to a limit).

1.5.3 Manipulating the FD table

The kernel provides syscalls to manipulate FDs without affecting the underlying open file:

- `close(fd)` – removes `fd` from the table. If this was the last reference, the underlying open file description is destroyed.
- `dup(fd)` – returns a new FD pointing to the same open file as `fd`. They share offset, flags, etc.
- `dup2(oldfd, newfd)` – makes `newfd` a duplicate of `oldfd`, closing whatever was at `newfd` first. The classical use: redirecting `stdin/stdout`.
- `dup3(oldfd, newfd, flags)` – like `dup2` with an explicit flags argument.
- `fcntl(fd, F_DUPFD, ...)` – like `dup` but with control over the resulting FD number.

How a shell implements `my_program > out.txt`:

1. `fork()` – create a child.
2. In the child: `open("out.txt", O_WRONLY | O_CREAT, 0644)` – get a FD pointing to the file (say, FD 3).
3. In the child: `dup2(3, 1)` – make FD 1 (`stdout`) point to the file too. The original `stdout` is closed.
4. In the child: `close(3)` – the extra FD is no longer needed.
5. In the child: `execve("/usr/bin/my_program", ...)` – run the program, which writes to FD 1 as usual, except FD 1 now goes to the file.

1.5.4 Why FDs matter for behavioral analysis

A single syscall like `read(7, buf, 1024)` is ambiguous: FD 7 could be a file, a socket, a pipe, or anything else. To understand what the syscall *means*, you need to track which FDs are which kind of object.

Tracing tools handle this by maintaining state:

- Watch `open`, `openat` – creates file FDs.
- Watch `socket`, `accept`, `accept4` – creates socket FDs.
- Watch `pipe`, `pipe2` – creates pipe FDs.

- Watch `epoll_create*`, `eventfd`, etc. – creates event-source FDs.
- Watch `dup`, `dup2`, `dup3`, `fcntl(F_DUPFD)` – duplicates existing FDs.
- Watch `close` – frees FDs.
- Maintain a per-process map from FD number to “what is this”.

`/proc/<pid>/fd/` provides a shortcut: each FD is a symlink to its target. For files, the symlink resolves to the path. For sockets, it resolves to `socket:[inode]`. For pipes, `pipe:[inode]`.

Further reading

- Kerrisk, *TLPI*, Ch. 4–5 (File I/O), 56 (Sockets), 44 (Pipes).
- [Beej's Guide to Network Programming](#) – §1–5 explain sockets as FDs.

1.6 ptrace: The Classic Tracing Mechanism

1.6.1 What ptrace does

`ptrace` (“process trace”) is the syscall that lets one process inspect and control another. It’s the foundation of `strace`, `gdb`, and most pre-eBPF tracing tools. Despite its single name, it’s really a multiplexed interface – one syscall with many different “requests” that do different things.

The basic model: a *tracer* attaches to a *tracee*. While attached, the tracer can stop the tracee at various points, examine its state, modify it, and resume execution.

Important requests:

- `PTRACE_TRACEME` – called by a process to indicate “my parent will trace me”. Used at startup for debuggers that launch the debuggee.
- `PTRACE_ATTACH` – attach to an already-running process. Requires either being the parent, having the same UID, or being root (with capabilities).
- `PTRACE_SEIZE` – a newer variant that attaches without sending a stop signal.
- `PTRACE_DETACH` – stop tracing the tracee, leaving it running normally.
- `PTRACE_CONT` – resume a stopped tracee, run until the next signal or event.
- `PTRACE_SYSCALL` – resume until the tracee enters or exits a syscall, then stop.
- `PTRACE_SINGLESTEP` – execute one instruction, then stop.
- `PTRACE_GETREGS` / `PTRACE_SETREGS` – read or write the tracee’s registers.
- `PTRACE_PEEKDATA` / `PTRACE_POKEDATA` – read or write the tracee’s memory, one word at a time.
- `PTRACE_PEEKTEXT` / `PTRACE_POKETEXT` – aliases for the above (legacy).
- `PTRACE_GETSIGINFO` / `PTRACE_SETSIGINFO` – inspect or modify pending signals.

1.6.2 How strace works

When you run `strace ls`, here’s what’s happening underneath:

1. `strace` calls `fork()`.
2. The child calls `ptrace(PTRACE_TRACEME, ...)` to declare itself traceable.

3. The child calls `execve("/bin/ls", ...)`. This triggers a SIGTRAP that stops the child immediately.
4. The parent (strace) calls `wait()`, which returns with the child stopped.
5. Strace enters a loop:
 - (a) `PTRACE_SYSCALL` to resume the child until next syscall entry.
 - (b) `wait()` – child stops on syscall entry.
 - (c) `PTRACE_GETREGS` – read registers. The syscall number is in `rax` (on x86-64) and arguments are in the standard registers.
 - (d) If any argument is a pointer to a string or struct, use `PTRACE_PEEKDATA` to read the actual data from the child's memory.
 - (e) Print the syscall name and arguments.
 - (f) `PTRACE_SYSCALL` to resume until syscall exit.
 - (g) `wait()` – child stops on syscall exit.
 - (h) Read `rax` again – now it contains the return value.
 - (i) Print the return value.
6. When the child eventually exits, the loop terminates.

This means `strace` interrupts the tracee *twice per syscall* (entry and exit), and each interruption involves multiple `ptrace` calls. The overhead is substantial.

1.6.3 Limitations of ptrace

`ptrace` has fundamental limitations that motivated newer tools:

One tracer at a time. A process can be ptraced by at most one tracer. If `gdb` is attached, `strace` can't also attach. If you have an automated tracing tool running system-wide, no one can debug.

Very high overhead. Every syscall causes:

- Two context switches (entry stop, exit stop).
- Two `wait` calls in the tracer.
- Multiple `ptrace` calls in the tracer to read state.
- If argument data is needed: more `ptrace` calls, each reading 8 bytes from the tracee's memory.

A program traced under `strace` typically runs 10–100x slower than untraced. For some workloads it can be 1000x slower.

The tracee runs interrupted. Each syscall pauses the tracee while the tracer processes. This can:

- Change scheduling behavior. Bugs that only appear under timing pressure may not reproduce.
- Cause network timeouts. A traced server may miss heartbeat windows.

- Affect concurrent code in subtle ways.

Trivially detected. A process can detect that it's being traced:

- Call `ptrace(PTRACE_TRACEME, ...)` on itself. If it succeeds, no one's tracing. If it returns `EPERM`, someone is.
- Read `/proc/self/status` and check the `TracerPid` line. Non-zero means traced.
- Time some operations. Some kinds of tracing add detectable timing artifacts.

Malware uses these checks routinely.

Need elevated permissions. Attaching to a process you didn't start requires either the same UID or `CAP_SYS_PTRACE`. On many modern Linux systems, this is restricted by default via `kernel.yama.ptrace_scope`.

These limitations are exactly why eBPF emerged as the modern alternative.

Going deeper

The single-tracer limitation comes from the fact that `ptrace` fundamentally couples the tracee to a specific tracer process. The tracee can't be paused by two different "stop" commands; it can't deliver state to two different observers; signals and exit notifications go to one parent.

eBPF sidesteps this by being passive instead of active. eBPF programs observe events; they don't pause the observed process. Many eBPF programs can attach to the same event simultaneously. This is the architectural change that made system-wide, low-overhead tracing practical.

Further reading

- Eli Bendersky, "[How Debuggers Work: Parts 1, 2, 3](#)" – the best free `ptrace` tutorial.
- `man 2 ptrace` – long but essential reference. Focus on the request types above.
- Pradeep Padala, "Playing with `ptrace`" in Linux Journal – practical walkthrough.

1.7 eBPF: Modern Tracing Infrastructure

1.7.1 What eBPF is

eBPF (extended Berkeley Packet Filter) is a virtual machine that runs inside the Linux kernel. User-space programs can load eBPF programs (written in a restricted dialect of C, compiled to eBPF bytecode) into the kernel, where they execute in response to kernel events: a syscall is entered, a network packet arrives, a kernel function is called, a tracepoint fires.

The crucial insight: eBPF programs run *in the kernel*, so they can observe events without any context switch. They can also filter and aggregate events in-place, so only the relevant data ever leaves the kernel for user-space consumption. This eliminates the per-event overhead that crippled `ptrace`-based monitoring.

The name is historical. The original BPF (Berkeley Packet Filter, from 1992) was a small VM that ran network packet filtering rules in the kernel for tools like `tcpdump`. Around 2014, Alexei Starovoitov extended BPF substantially (more instructions, more data types, ability to attach to events beyond network packets) and the result was called "extended BPF" or eBPF. The original BPF is now retroactively called "classic BPF".

1.7.2 The verifier: why this is safe

Letting userspace upload arbitrary code into the kernel sounds dangerous. The reason it's safe is the *eBPF verifier*: a static analyzer that runs before any eBPF program is loaded. The verifier proves that the program:

- Always terminates. Originally, no loops were allowed at all; newer kernels allow bounded loops where the verifier can prove an upper bound on iterations.
- Only accesses memory it's authorized to access: its own stack (a small 512-byte allocation), BPF maps (data structures explicitly allocated for sharing with userspace), and context provided by the kernel (the event data).
- Never dereferences a NULL pointer or an uninitialized pointer. The verifier tracks pointer provenance through the program.
- Doesn't leak kernel pointers to userspace.
- Doesn't violate type rules.

If the verifier can't prove safety, the program is rejected with an error message. This is why writing eBPF can be frustrating – you can't just write any C code; you have to write code the verifier can analyze. But it's also why eBPF is trustworthy enough to run in the kernel.

The verifier is genuinely impressive. It can track which fields of a struct have been read, which pointers have been NULL-checked, which arithmetic operations are bounded. It rejects programs that would cause undefined behavior on any input.

1.7.3 Hook types: where programs can attach

eBPF programs attach to specific events in the kernel. The main hook types relevant to tracing:

Tracepoints. Stable, named hook points that kernel developers have explicitly placed at semantically meaningful spots. Example: `tracepoint:syscalls:sys_enter_execve` fires every time the kernel is about to execute the `execve` syscall. The arguments are available in a structured form.

Advantages: stable across kernel versions, well-documented, available for every syscall.

Disadvantages: only exists where kernel developers added one. For internal kernel functions you want to observe, you'd need a kprobe.

kprobes (Kernel Probes). Dynamically inserted breakpoints at arbitrary kernel function addresses. Example: `kprobe:vfs_read` fires every time the kernel's `vfs_read` function is called.

Advantages: can probe any function, no need for the function to be “instrumented” in advance.

Disadvantages: less stable across kernel versions (function signatures and even names change between releases). Arguments must be decoded manually.

kretprobes. Like kprobes but trigger on function return. Useful when you want to see return values.

uprobes / uretprobes. Same idea but for user-space functions. You can probe any function in any binary. `uprobe:/usr/bin/python3:Py_Main` would fire on every call to Python's main function.

USDT (User Statically Defined Tracing). Stable user-space tracepoints that application developers can embed in their code. Databases and language runtimes often have these.

Perf events. Hardware performance counters and CPU events.

XDP (eXpress Data Path). Network packet handling at the lowest possible level (in the driver, before any kernel networking processing). Used for high-performance packet filtering, DDoS protection.

LSM (Linux Security Module) hooks. A relatively recent addition that lets eBPF programs make security decisions (allow / deny) at LSM points in the kernel.

For behavioral malware detection, the most important hooks are syscall tracepoints (for the syscall interface) and kprobes/uprobes (for finer-grained or user-space events).

1.7.4 BPF maps: state across invocations

eBPF programs are stateless: each invocation runs in isolation. To accumulate data over time, they use *BPF maps* – kernel-allocated data structures shared between the eBPF program (in kernel space) and the user-space tool that loaded it.

Common map types:

- `BPF_MAP_TYPE_HASH` – generic key-value hash table.
- `BPF_MAP_TYPE_ARRAY` – integer-indexed array of values.
- `BPF_MAP_TYPE_PERCPU_*` – per-CPU versions of hash and array maps. No locks needed; userspace aggregates by summing across CPUs. Vital for performance.
- `BPF_MAP_TYPE_LRU_HASH` – hash map that automatically evicts least-recently-used entries when full.
- `BPF_MAP_TYPE_HISTOGRAM` – specialized for counting into buckets, useful for latency distributions.
- `BPF_MAP_TYPE_RINGBUF` – a ring buffer for streaming events from kernel to userspace.
- `BPF_MAP_TYPE_PERF_EVENT_ARRAY` – earlier ring-buffer mechanism, still common.
- `BPF_MAP_TYPE_PROG_ARRAY` – contains other BPF programs, enabling tail calls.
- `BPF_MAP_TYPE_STACK_TRACE` – captured stack traces for sampling.

Typical flow: the BPF program runs on each event, updates a map. Userspace periodically reads the map for aggregated statistics, or streams events from a ring buffer in real time.

1.7.5 bpftrace, bcc, libbpf: three ways to write eBPF tools

bpftrace. A high-level scripting DSL inspired by AWK and DTrace. Best for ad-hoc tracing and exploration. Example: count `execve` calls by process name, refreshed every 5 seconds.

```
sudo bpftrace -e '
tracepoint:syscalls:sys_enter_execve {
    @execs[comm] = count();
}
interval:s:5 {
    print(@execs); clear(@execs);
},'
```

Five lines, no compilation, instant feedback. `bpfftrace` compiles your script to BPF bytecode behind the scenes, loads it, attaches it, and prints results. It's ideal for sketching ideas.

bcc (BPF Compiler Collection). A Python frontend that lets you embed C-written BPF programs in Python scripts and process events in Python. More verbose than `bpfftrace` but more flexible – you can do complex event processing in Python while keeping in-kernel hooks lightweight.

The `bcc-tools` package ships dozens of ready-made tools that are excellent reference implementations: `execsnoop` (trace all executions), `opensnoop` (trace all file opens), `tcpconnect` (trace all TCP connections), `biosnoop` (trace block I/O), and many more. Reading the source of these tools is one of the best ways to learn eBPF.

libbpf. The modern, low-level C library. You compile BPF programs once with clang (which has an eBPF backend), package them as ELF files, and load them at runtime with `libbpf`. The result has no Python or interpreter dependency, so it's suitable for production tools.

For learning and experimentation, `bpfftrace` is ideal. For production monitoring tools, `libbpf` is the modern standard.

1.7.6 Why eBPF wins for behavioral monitoring

Compared to `ptrace`, eBPF offers:

Overhead. 10–100x lower. Programs run in-kernel without context switches. A trace that's prohibitive under `ptrace` becomes routine under eBPF.

In-kernel filtering. You can decide at the source of an event whether to log it. Without this, every syscall in the system would have to be sent to userspace just for the tracer to decide it wasn't interesting.

Multiple consumers. Many BPF programs can attach to the same event. Several monitoring tools can run simultaneously; you can debug a process being monitored.

In-kernel aggregation. Build histograms, counters, top-N maps inside the kernel. Userspace just reads summaries.

Undetectable by tracee. eBPF doesn't appear in the process's `TracerPid`. A process can detect that eBPF programs are loaded system-wide (root can list them with `bpftool`), but not that it specifically is being observed.

Production-ready. Run on live production systems. Used in monitoring platforms (Datadog, New Relic), security tools (Falco, Tracee, Tetragon), networking infrastructure (Cilium, Katran), and more.

This is why every modern security monitoring tool is built on eBPF, and why your understanding of behavioral detection in 2025 has to start from eBPF rather than `ptrace`.

Try this

Useful `bpfftrace` one-liners to internalize the tool:

```
# All execve calls system-wide with command and program
```

```

sudo bpftrace -e '
tracepoint:syscalls:sys_enter_execve {
    printf("%s -> %s\n", comm, str(args->filename));
},

# File-open counts per process over 10 seconds
sudo bpftrace -e '
tracepoint:syscalls:sys_enter_openat {
    @[comm] = count();
}
interval:s:10 { exit(); }',

# Distribution of read sizes
sudo bpftrace -e '
tracepoint:syscalls:sys_enter_read {
    @reads = hist(args->count);
}
interval:s:30 { exit(); }',

# Track TCP connections
sudo bpftrace -e '
kprobe:tcp_connect {
    printf("%s connecting\n", comm);
},

```

Each is roughly five lines, no compilation step, runs immediately. Production behavioral monitoring tools are these same ideas at scale.

Further reading

- Brendan Gregg, *BPF Performance Tools* (2019). Chapters 1–5 are mandatory; rest is reference. *The eBPF book*.
- [bpftrace one-liner tutorial](#) – work through every example hands-on.
- [ebpf.io](#) – ecosystem landing page.
- Liz Rice, “What is eBPF?” on YouTube – best conceptual introduction.
- LWN.net articles on BPF – search “BPF for the busy person”. LWN is the definitive source for kernel-level coverage.
- bcc-tools source on GitHub – read several of the ready-made tools to learn idioms.

1.8 Foundational Papers for Syscall-Based Detection

1.8.1 Forrest et al. 1996: “A Sense of Self for Unix Processes”

This is the founding paper of syscall-based intrusion detection. The conceptual move was to define a notion of “normal” for a process based on its syscall sequences during legitimate execution, and flag deviations as “non-self”.

The model:

1. For a given process under normal operation, collect traces of its system calls.
2. Extract all n -grams: sliding windows of n consecutive syscalls. The set of distinct n -grams observed during training is called the *self* of this process.
3. During monitoring, examine each n -gram in the live execution. If it’s in the self set, normal. If not, it’s a mismatch – an “anomaly”.

4. Count anomalies over a window. If many occur in a short window, raise an alarm.

The paper demonstrated this on **sendmail** and other server processes. Notable results:

- Self sets are surprisingly small. After collecting enough training data, the rate of new n -grams plateaus.
- Anomalies during attacks are clustered, not scattered. An intrusion produces a burst of unfamiliar syscall sequences.
- The technique is robust to small n . They found $n = 6$ worked well; larger n gave diminishing returns.

This was published in 1996. The framework – learn normal, flag abnormal – has been the spine of behavioral detection ever since. Every subsequent paper either builds on this approach (different feature representations, different anomaly metrics) or rejects it (and proposes supervised classification instead, with malicious-class training data).

Key idea

Why this matters: the same n -gram technique is competitive 30 years later. Many papers comparing fancy deep learning approaches with the Forrest baseline find the baseline wins or ties on real-world test sets. This is a cautionary tale about complexity-for-its-own-sake in security ML – and a reason to always include a Forrest-style baseline in any new method evaluation.

1.8.2 Forrest, Warrender, Pearlmutter 1999: “Detecting Intrusions Using System Calls: Alternative Data Models”

The follow-up paper. Compares four feature models for syscall sequences:

1. **Frequency.** Just counts of each syscall in a window. No ordering.
2. **Sequences (n-grams).** As in the 1996 paper. Captures local ordering.
3. **Sequence covers / lookahead pairs.** A relaxed n -gram: “syscall A is followed by syscall B within k positions, ever”. Less sensitive to permutations.
4. **Hidden Markov Models (HMMs).** Probabilistic models of syscall sequences. Theoretically more expressive.

The key result: simple n -grams (length 6) work nearly as well as HMMs at far lower complexity. This is a deep finding – the simplest sequence-aware model is enough for most detection tasks.

For your work building behavioral classifiers, this suggests a methodology:

1. Always include a frequency-only baseline.
2. Always include an n -gram-based baseline.
3. Only if you can beat the n -gram baseline does fancier modeling pay off.

This is exactly the kind of methodological discipline that the Arp et al. 2022 paper (covered in Track 4) recommends, and that’s missing from much published security ML.

Further reading

- Forrest, Hofmeyr, Somayaji, Longstaff, “A Sense of Self for Unix Processes”, IEEE S&P 1996.

- Forrest, Warrender, Pearlmutter, “Detecting Intrusions Using System Calls: Alternative Data Models”, IEEE S&P 1999.

1.9 Track 1 Self-Check Questions

If you can answer these confidently from memory, you have Track 1 cold:

1. What does `strace -f -e trace=network curl example.com` show, and what does it miss? (Hint: vDSO; anything in kernel that doesn't pass through the syscall boundary.)
2. Why do syscall numbers differ between x86-64 and ARM, and what does that imply for a behavioral classifier trained on one architecture and deployed on another?
3. Walk through what happens, in syscalls, between `./myprogram` on a shell and the first instruction of `main()`.
4. What's the difference between `fork()` and `clone()`, and which does `pthread_create` use?
5. Why is eBPF preferable to `ptrace` for production behavioral monitoring? Name at least three reasons.
6. If a process is in D state (uninterruptible sleep), what does `SIGKILL` do?
7. What's the `TracerPid` field in `/proc/<pid>/status`, and how do malware authors use it?
8. Why is `mprotect(..., PROT_EXEC)` on a previously non-executable page suspicious?
9. What's the difference between a section and a segment in an ELF file?
10. What does the PLT/GOT mechanism do, and why is the GOT a common exploit target?

2 IoT and Embedded Systems

IoT devices aren't "small Linux". They have specific architectures, filesystems, init systems, library choices, economic constraints, and threat models that make them their own domain. To analyze IoT firmware or detect IoT malware, you need to understand what makes IoT IoT. This track covers the distinguishing features of embedded Linux systems and the tooling that has evolved around them.

2.1 What "IoT" Means in This Context

2.1.1 The spectrum of embedded systems

"IoT" is a notoriously broad term. It can refer to anything from a 25-cent microcontroller in a temperature sensor to a fully-equipped industrial controller. For the kind of malware-detection work covered in this guide, the relevant slice is *Linux-based IoT*: devices with enough resources to run Linux, capable of being targeted by Mirai-class network malware. The spectrum looks like this:

Bare-metal microcontrollers. Devices like Arduino-class boards, simple sensors, basic remotes. No OS, or a thin runtime. Targeted by malware essentially never – not enough attack surface and not enough capability to be useful in a botnet. Out of scope.

RTOS-based devices. Microcontrollers running a real-time operating system like FreeRTOS, Zephyr, ThreadX. Most automotive ECUs, many medical devices, smart-home microcontrollers (ESP32-based devices running ESP-IDF). Some research exists on RTOS malware, but the ecosystem is fragmented and small. Out of scope for the mainstream IoT malware discussion.

Linux-based IoT. Where the action is. Devices typically have:

- A 32-bit or 64-bit CPU (ARM or MIPS, usually).
- 32–512 MB of RAM.
- 8–256 MB of flash storage.
- A network interface (Ethernet or WiFi, often both).
- A stripped-down Linux distribution.

This is the category that includes routers, switches, IP cameras, DVRs, NAS appliances, set-top boxes, smart-home hubs, single-board computers (Raspberry Pi class), industrial gateways.

Edge / gateway devices. Higher-end systems with more RAM and storage, sometimes running x86 or 64-bit ARM, sometimes running full distributions like Ubuntu Core or Yocto-based builds. Examples: industrial PCs, edge-compute boxes, fancy gateways. The line between "edge device" and "small server" is blurry.

The dominant targets of public IoT malware – Mirai and its descendants, Gafgyt, Mozi – are Linux-based IoT. These devices have enough capability to host real malware, are widely deployed, are always online, are rarely patched, and frequently ship with default credentials or known-vulnerable software. They are essentially the Internet's underbelly.

2.1.2 Why the security is bad: structural economics

The security state of IoT is not bad by accident. It results from incentives baked into the industry:

Tiny margins. A \$30 home router might have only \$3 of margin after manufacturing, distribution, and retail. There’s no slack budget for proper security engineering. Adding a security audit cost or paying for a separate firmware build with mitigations enabled could cut margins in half.

One-shot revenue model. The vendor’s commercial relationship with the customer typically ends at purchase. Producing patches takes engineering time and produces no revenue. Many vendors don’t even have a coherent patch distribution mechanism.

Long device lifetimes. A router or camera bought in 2015 may still be deployed a decade later, running 2015’s kernel and 2015’s userspace components. Public CVEs that were patched on desktop Linux years ago remain exploitable.

Reference designs. Most IoT vendors don’t actually develop their own firmware from scratch. They license reference designs from SoC vendors (Realtek, MediaTek, Broadcom, Qualcomm) and lightly customize them. A vulnerability in a reference design propagates to dozens of consumer products. The 2022 “Realtek SDK” family of bugs affected millions of devices across many brands – because all the brands were shipping variations of the same Realtek-provided code.

Supply-chain opacity. The vendor you buy from may have rebadged a product from an Original Design Manufacturer (ODM) who sourced components from yet another supplier. Tracking which products contain which underlying software is difficult. The SBOM (Software Bill of Materials) movement is trying to fix this, but adoption is slow.

No accountability for breaches. When IoT devices are weaponized into a botnet that DDoSes a third party, the cost is borne by the victim, not the vendor of the compromised devices. There’s no market mechanism punishing vendors for shipping vulnerable products.

Key idea

This is the world that behavioral detection is meant to address. Signature-based approaches assume vendors will distribute updated signatures. Vendors don’t. Device-local behavioral detection – a device that watches its own behavior for anomalies and acts on what it sees – doesn’t depend on the supply chain caring. It’s a model of security that aligns with the actual incentive structure.

This is why the problem is interesting beyond academic curiosity: it’s a place where the engineering can be *decoupled* from the broken business model.

Further reading

- Costin, Zaddach, Francillon, Balzarotti, “A Large-Scale Analysis of the Security of Embedded Firmwares” (USENIX Security 2014) – the foundational empirical paper on what’s actually shipped in firmware.
- [OWASP IoT Top 10](#) – short threat-vocabulary primer.
- DEF CON IoT Village playlist on YouTube – the culture of the field.

2.2 Architectures

2.2.1 The architecture zoo

Unlike the desktop and server world (where x86-64 dominates almost completely), IoT is architecturally diverse. The major players you’ll encounter:

ARM. The most common in IoT. “ARM” refers to a family of architectures developed by Arm Ltd.; the company licenses their designs to many chip manufacturers (Broadcom, Qualcomm, MediaTek, Allwinner, NVIDIA, Apple, and many others). Relevant subfamilies:

- **ARMv7-A (32-bit, with MMU)** – the classic “Linux on ARM” chip class. Cortex-A7, A9, A53 (when run in 32-bit mode). Older / cheaper routers, lots of consumer IoT, many single-board computers.
- **ARMv8-A / AArch64 (64-bit)** – newer routers, single-board computers, premium consumer devices. Cortex-A53 (64-bit), A55, A72.
- **ARMv7-M / ARMv8-M (no MMU, microcontroller-class)** – Cortex-M0, M3, M4, M7. These run RTOS or bare metal, not Linux. Out of our scope.

ARM CPUs are bi-endian – the hardware supports both little-endian and big-endian modes, configurable at boot. Linux on ARM is almost always little-endian (**armel** or **aarch64-el**). You may rarely encounter big-endian ARM (**armeb**) in legacy network equipment.

MIPS. Heavily used in routers, especially older ones and devices targeting the Asian market. MIPS is also bi-endian, and both endiannesses are common in the wild:

- **MIPS32** (32-bit) – the default for most MIPS-based routers.
- **MIPS64** (64-bit) – rare in consumer IoT.
- **MIPSEL** – little-endian. More common in newer MIPS hardware.
- **MIPSEB** – big-endian. More common in older Broadcom-based routers.

A peculiarity of MIPS: the same chip can be either endianness depending on a strap pin or firmware config. You can have two routers with identical SoCs but different endianness. Any tooling for IoT analysis has to support both.

x86 and x86-64. Rare in true consumer IoT but common in industrial PCs, NAS appliances, edge gateways, and a few high-end home devices. Analysis is easier because all standard tooling works natively.

Others.

- **PowerPC** – legacy network equipment (Cisco enterprise gear, some industrial controllers).
- **SuperH (SH4)** – mostly legacy Japanese consumer electronics. Some old set-top boxes still use it.
- **RISC-V** – the future. Currently rare in deployed IoT but rising. Expect more devices over the next 5–10 years.
- **Xtensa** – Tensilica’s customizable cores, used by Espressif in ESP8266 / ESP32 (which run RTOS, not Linux).

2.2.2 The cross-architecture problem

This architectural diversity is the central pain point of IoT analysis. Consider what changes between architectures:

- **Instruction set.** Different instructions, different mnemonics, different lengths. Disassembly tools need separate backends per architecture.

- **Calling conventions.** Different registers used for arguments, return values, syscall numbers.
- **Syscall numbers.** As discussed in Track 1, the same syscall has different numbers across architectures.
- **ABI quirks.** Argument alignment rules, struct packing rules, varargs handling all differ.
- **Endianness.** Even on the same instruction set, byte order can flip.
- **Memory layout.** Page sizes, kernel/user split, ASLR amounts all vary.

For dynamic analysis at the syscall level, the right approach is to normalize: extract syscall *names* rather than numbers, and analyze them architecture-independently. Most tracing tools do this for you (**strace** prints names, not numbers).

For static analysis, you often need separate workflows per architecture. Ghidra handles all the major architectures, but you have to load each binary with the right backend.

For ML-based detection systems, cross-architecture generalization is an open problem. A detector trained on x86 syscall traces may not transfer to ARM traces, even after name normalization, because the *distributions* of syscall use differ between architectures (libc implementations are different, system call patterns of common utilities like BusyBox vs glibc-based tools differ).

2.2.3 What you need to know about each architecture

You don't need to be fluent in MIPS assembly to do meaningful IoT analysis. You need to:

- Recognize each on sight. **file binary** tells you instantly: “ELF 32-bit LSB executable, ARM, EABI5” or “ELF 32-bit MSB executable, MIPS, MIPS32”.
- Know how to disassemble. Ghidra handles all of these out of the box. Just open the file and let it auto-detect.
- Know how to emulate. QEMU supports each: **qemu-arm-static**, **qemu-aarch64-static**, **qemu-mips-static**, **qemu-mipsel-static**, **qemu-ppc-static**, etc.
- Understand the syscall ABI differences (covered in Track 1).

For deeper ARM literacy – which is genuinely useful given how dominant ARM is – the Azeria Labs tutorials are the standard free resource.

Try this

Download a few firmware images from different vendors. Try OpenWrt builds for different targets (downloads.openwrt.org has many, organized by target architecture). Extract each with **binwalk -e**. Pick the BusyBox binary from each and run **file** on it. You'll see the breadth of architectures in real-world IoT:

- “ELF 32-bit LSB executable, ARM, EABI5” – a Cortex-A device.
- “ELF 64-bit LSB executable, ARM aarch64” – a newer device.
- “ELF 32-bit MSB executable, MIPS, MIPS32” – big-endian MIPS (older Broadcom).
- “ELF 32-bit LSB executable, MIPS, MIPS32” – little-endian MIPS.
- Rarely: “ELF 32-bit MSB executable, PowerPC” – legacy network equipment.

Further reading

- [Azeria Labs ARM Assembly tutorials](#) – parts 1–5.
- Wikipedia entries on MIPS architecture and ARM architecture – sufficient overviews.
- `man 2 syscall` – the per-architecture syscall ABI table.

2.3 Firmware Structure

2.3.1 What’s in a firmware blob

A typical IoT firmware image, as downloaded from a vendor’s website, is a single binary file (`.bin`, `.img`, sometimes `.trx`, sometimes `.zip` containing multiple files). When the device flashes it, the contents are written to the device’s onboard flash memory, partitioned roughly as:

1. **Bootloader** (typically U-Boot) – a few hundred KB. Initializes hardware, then loads the kernel.
2. **Kernel image** (compressed) – a few MB. The Linux kernel itself.
3. **Device tree blob (DTB)** – a few KB on ARM devices. Describes the hardware to the kernel.
4. **Root filesystem** – the bulk of the image, typically tens of MB. A SquashFS, JFFS2, or similar filesystem containing `/bin`, `/sbin`, `/etc`, etc.
5. **Configuration partition (NVRAM)** – a small writable area for device-specific settings.
6. Possibly additional partitions: a second rootfs for failover, a recovery partition, vendor-specific data.

Vendors typically wrap this with their own header format (with a checksum, version number, and optionally a signature) for OTA updates. The header lets the device verify it’s flashing the right version. Tools like `binwalk` know about most common vendor formats.

2.3.2 Bootloaders

The bootloader is the first software to run when the device powers on. Its job: initialize the most critical hardware (DRAM controller, basic clocks), then load the kernel from flash and jump to it. On Linux-based IoT devices the dominant bootloader is U-Boot.

U-Boot. Open source, configurable per-platform. Features:

- Initial hardware setup (DRAM controller, clocks, basic peripherals).
- A console, typically on UART, where you can interact during boot.
- Commands to load kernel and rootfs from various sources: flash, network (TFTP, NFS), USB, SD card.
- Environment variables stored in flash: `bootcmd` (what to run at boot), `bootargs` (kernel command line), `ipaddr`, `serverip`, etc.
- Optional secure boot integration on devices that support it.

U-Boot is interesting for security research because access to the console – physically, via UART pads on the PCB and a USB-to-UART adapter – often lets you bypass the normal boot flow. You can boot into a recovery mode, boot a custom kernel image from TFTP, or extract the running kernel and rootfs over the network. Many IoT vulnerability research workflows start with “find the UART pads, get a U-Boot prompt”.

Proprietary bootloaders. Some SoC vendors have their own:

- Broadcom CFE (Common Firmware Environment) – found on Broadcom-based routers.
- Realtek’s various bootloaders.
- MediaTek’s various bootloaders.

Conceptually similar to U-Boot but less standardized. Documentation is often poor or proprietary.

2.3.3 Filesystem formats

The root filesystem on an IoT device is typically a compressed read-only image (for size and integrity) or a writable flash-friendly filesystem. The main formats:

SquashFS. The dominant rootfs format for read-only filesystems on IoT. Highly compressed (typically with LZMA, XZ, or Zstd), small block size optimized for low-resource devices. Created on a build machine with `mksquashfs`, mounted read-only at runtime, extracted offline with `unsquashfs`.

You’ll work with SquashFS constantly. Workflow:

```
binwalk -e firmware.bin
# binwalk extracts what it finds; SquashFS images become .squashfs
# files
cd _firmware.bin.extracted/
ls -la *.squashfs                # find the rootfs
unsquashfs squashfs-root.bin     # creates squashfs-root/ directory
cd squashfs-root
ls -la                          # browse like a normal Linux
                                # filesystem
```

JFFS2 (Journaling Flash File System v2). A writable filesystem designed for raw NAND or NOR flash. Provides wear leveling (spreading writes across the flash to prevent burning out cells). Was the dominant writable filesystem in the 2000s. Less common for rootfs in modern devices but still used for writable partitions (configuration, logs).

UBIFS. The modern successor to JFFS2 for flash. Better performance, more sophisticated wear leveling. Used in newer devices, especially those with larger flash.

CramFS. Older read-only compressed filesystem. Less efficient than SquashFS; mostly obsolete but occasionally seen in legacy devices.

YAFFS / YAFFS2. “Yet Another Flash File System”. Used in some devices, especially Android-based older ones.

ext4. The standard Linux filesystem. Found on more capable IoT devices, especially SD-card-based ones (Raspberry Pi, some smart speakers). No special handling needed – it mounts like any ext4.

cpio (initramfs / initrd). Sometimes the entire rootfs of a small device is an initramfs – a cpio archive loaded into memory by the kernel at boot. Extract with `cpio -idmv < initrd`.

2.3.4 Device Tree

ARM-based Linux uses device tree files to describe hardware to the kernel. Before device tree, kernels had board-specific code hardcoded for every supported device. With device tree, the same kernel can run on any board if you provide the right description.

- `.dts` (Device Tree Source) – human-readable description in a tree-structured syntax.
- `.dtb` (Device Tree Blob) – compiled binary form, loaded into memory at boot.
- `dtc` (Device Tree Compiler) – the compiler.

For security research, device tree files are informative – they describe the SoC, RAM size, attached peripherals (sensors, USB, NICs), GPIO pins, and so on. Converting a DTB back to readable source: `dtc -I dtb -O dts device.dtb -o device.dts`.

2.3.5 NVRAM

NVRAM (non-volatile RAM) on routers and similar devices stores configuration: WiFi SSID and password, admin credentials, port forwarding rules, DDNS settings. Implementation varies considerably:

- On some devices, NVRAM is a dedicated flash partition with a custom key-value format.
- On others, it's a regular file in a writable partition, accessed via a vendor library.
- Broadcom devices typically have a `libnvram.so` library exposing `nvram_get()` and `nvram_set()` functions.
- Some devices use a small SQLite database.

Key idea

NVRAM is one of the biggest reasons firmware emulation is hard. When emulated firmware boots, it has no real NVRAM – there's no flash partition full of vendor defaults waiting. Programs that call `nvram_get("wan_ip")` get NULL, and either crash on the assumption that it should always succeed, or fall into error paths that prevent normal operation.

Modern emulators like FirmAE and EMBA include fake `libnvram.so` libraries that intercept these calls and return plausible default values. This is the heart of the “arbitrated emulation” technique introduced in the FirmAE paper.

Further reading

- Costin et al. 2014 – Section 4 on firmware composition.
- [Embedded Linux Wiki](#) – many entries on rootfs and bootloader structure.
- [U-Boot manual](#) – skim. Goal is recognition, not mastery.
- Wikipedia entries on SquashFS, JFFS2, UBIFS – adequate overviews.

2.4 IoT Userspace Conventions

2.4.1 BusyBox: the heart of small Linux userspaces

The single most distinctive feature of IoT Linux userspaces is BusyBox. BusyBox is a single binary that implements over 300 Unix utilities (`ls`, `cat`, `grep`, `sh`, `ifconfig`, `wget`, `telnetd`, `httpd`, and many more) in a few hundred KB. Each utility is called an “applet”.

How it works: at link time, the user picks which applets to compile in. The resulting single binary can be invoked as any of them. The dispatch happens via the program name (`argv[0]`). In a typical IoT rootfs:

```
$ ls -la /bin/
lrwxrwxrwx 1 root root      7 ... ls -> busybox
lrwxrwxrwx 1 root root      7 ... cat -> busybox
lrwxrwxrwx 1 root root      7 ... grep -> busybox
lrwxrwxrwx 1 root root      7 ... sh -> busybox
... (hundreds more symlinks)
-rwxr-xr-x 1 root root 754320 ... busybox
```

When you run `ls /tmp`, the kernel actually execs `/bin/busybox` with `argv[0] = "ls"` and `argv[1] = "/tmp"`. BusyBox's `main()` examines `argv[0]`, looks up the corresponding applet's main function, and calls it.

This design saves enormous space – 300 utilities in one 750 KB binary instead of 300 separate binaries totaling tens of MB. For a router with 16 MB of flash, this is the difference between possible and impossible.

Key idea

Implications for behavioral analysis:

- In `ps` output, you may see “busybox” or the applet name depending on configuration. Track both.
- Per-binary fingerprinting doesn't work as on desktop – every utility shares one binary.
- To know which applet is running, look at `/proc/<pid>/cmdline` (gives `argv`) rather than `/proc/<pid>/exe` (always points to busybox).
- BusyBox itself is what malware competes with. When Mirai-class malware kills the device's existing telnet daemon to take over the port, the daemon it kills is often BusyBox's `telnetd` applet.

2.4.2 libc choices: glibc vs uClibc vs musl

The C standard library on desktop Linux is almost universally glibc (GNU C Library) – feature-rich and large (3 MB on disk per copy, more in memory). On embedded devices, smaller alternatives are common:

uClibc and uClibc-ng. The classic embedded libc. Designed for minimum footprint – a build might be 300 KB compared to glibc's 3 MB. Omits some features (locale support, full NSS modules). Dominant on older OpenWrt builds and many vendor firmwares from the 2000s and 2010s.

musl. A more modern alternative. MIT-licensed (vs glibc's LGPL), with a focus on correctness, standards compliance, and small size. Used by Alpine Linux and newer OpenWrt builds. Most modern embedded distributions are migrating to musl.

glibc. Found on more capable devices – ones with enough flash and RAM to spare. Many edge gateways and industrial devices use glibc.

Bionic. Android's libc. Found on Android-derived IoT devices.

For analysis, the libc choice has subtle effects:

- Function implementations vary, so the syscalls produced by a given libc call (`printf`, `malloc`, etc.) can differ.
- Symbol resolution and dynamic linking can differ.
- Some malware checks the libc to decide what techniques to use.
- Statically-linked malware avoids depending on the device's libc.

2.4.3 Init systems

The init system is the first user-space process to run, responsible for starting all others. On IoT devices the common choices:

sysvinit. Classic Unix init. Reads `/etc/inittab`, runs scripts in `/etc/init.d/`. Simple, well-understood, still used on many older devices.

procd. OpenWrt's modern init system. Simpler than `systemd`, designed for embedded constraints. You'll see this on many modern IoT devices, especially OpenWrt-based ones. Uses an event-driven model with `ubus` (a small RPC system) for service coordination.

systemd. The dominant desktop init system. Rare in true IoT because it's large (several MB of binaries plus extensive runtime data). You'll see it on more capable devices: some single-board computers, some industrial gateways, edge appliances.

Custom inits. Some vendors write their own init shell scripts or daemon. Less rigorous, often where IoT security goes sideways – subtle race conditions or missing cleanup in startup scripts have led to attacks.

For behavioral analysis, the init system is the root of every process tree. Understanding its conventions tells you what “normal” process trees look like on a particular device.

2.4.4 Web servers

Virtually every IoT device has a web admin interface. Common implementations:

BusyBox httpd. The simplest option, basic CGI support. Tiny binary footprint. Found in many low-end devices.

uhttpd. OpenWrt's slightly more capable web server. Supports CGI and Lua. Common in OpenWrt-based devices.

lighttpd. “Lightweight HTTPD”, genuinely lightweight. Used by mid-tier devices.

mini_httpd. Another tiny option.

Vendor-custom web servers. Many large vendors (D-Link, Netgear, Asus, etc.) have written their own web admin servers. These are often the most security-relevant components on the device – a substantial fraction of disclosed IoT CVEs are in vendor-custom web admin code.

The CGI scripts behind the admin interface are a classical goldmine for command-injection bugs:

- A parameter from a web form gets passed to a shell command via `system()` or `popen()` with insufficient sanitization.
- An attacker provides `; rm -rf /` (or similar) as the parameter value.
- The shell executes the injected command with whatever privileges the web server has – often root.

This pattern repeats so often that it’s basically a default assumption when looking at IoT firmware: “the web admin probably has command injection somewhere”.

2.4.5 Vendor daemons

Most devices run vendor-specific daemons for things like:

- Cloud telemetry – phoning home with usage data, often unencrypted or poorly secured.
- OTA update checking.
- Remote management – the TR-069 / CWMP protocol on port 7547 was historically used by ISPs to configure customer routers. Vulnerabilities in TR-069 clients have been exploited at scale.
- UPnP (Universal Plug and Play) service discovery – repeatedly vulnerable.
- Vendor mobile-app integration – proprietary protocols, often poorly secured.
- Cloud-relayed remote access (so users can access their devices from outside their LAN).

These are interesting because they’re often closed-source, have privileged network access (they need it to do their job), and are frequently the highest-attack-surface components on the device. Many recent IoT botnets propagate via vulnerabilities in vendor management daemons rather than the credential brute force Mirai used.

Further reading

- [BusyBox “About” page](#) – short overview.
- [OpenWrt documentation](#) – read the architecture overview and procd section.
- Wikipedia entries on musl and uClibc are adequate.

2.5 The IoT Attack Surface

2.5.1 Default credentials

The classic IoT attack vector, made famous by Mirai. The 2016 Mirai release included a hardcoded list of about 60 manufacturer default username/password pairs (`admin/admin`, `root/root`, `root/xc3511`, `root/vizxv`, and many more). The scanner connects to Telnet on port 23 or 2323 and tries each pair. If one works, the device is recruited into the botnet.

This still works at scale in 2025 for many devices. The 2016 Mirai attacks against KrebsOnSecurity (620 Gbps), OVH (1 Tbps), and the Dyn DNS infrastructure (which knocked out large parts of the US internet for hours) were generated entirely by default-credential IoT devices.

Some of the credentials in Mirai’s list weren’t random guesses: they were vendor-burned defaults that couldn’t be changed by users. “root/xc3511” was the default for certain Chinese DVR products; “root/vizxv” for certain cameras; and so on. The vendor-burned credentials are particularly insidious because no user-side action can mitigate them.

2.5.2 Internet-exposed services

Common services exposed by IoT devices, roughly ordered by danger:

- **Telnet (port 23, 2323)** – unencrypted, often password-only. The original Mirai vector. Should not be exposed on the Internet ever; still routinely is.
- **SSH (port 22)** – encrypted, but weak credentials still common. Better than Telnet but still abused.
- **HTTP admin (port 80, 8080)** – the web interface. Modern attacks often go through here via command injection or authentication bypass.
- **HTTPS admin (port 443, 8443)** – encrypted but the issues are usually in application logic rather than crypto.
- **UPnP (port 1900 for SSDP, port 5000 for SOAP)** – service discovery and configuration. Has had repeated serious vulnerabilities.
- **TR-069 / CWMP (port 7547)** – carrier management protocol. Was exploited at scale in 2016 against Deutsche Telekom routers in the so-called Mirai-Annie attack.
- **Various vendor management ports** (7000s, 8000s, 9000s ranges) – proprietary, often undocumented, often vulnerable.
- **DNS (port 53)** – if the device runs an open resolver (it shouldn't, but many do), it can be used in amplification DDoS attacks.
- **SMB (port 445)** – file sharing on NAS devices and some printers. Routinely vulnerable.
- **RTSP (port 554)** – video streaming on IP cameras. Cameras with default creds can be enumerated en masse.
- **MQTT (port 1883)** – IoT messaging protocol. Frequently configured without authentication.

2.5.3 Memory-corruption vulnerabilities in admin services

Even when default credentials aren't the issue, the admin services themselves frequently have classical memory-corruption bugs:

- Stack-based buffer overflows in C code parsing request parameters with unsafe functions (`strcpy`, `sprintf`, `strcat`).
- Heap overflows from unbounded reads or miscounted allocations.
- Format string vulnerabilities (less common now but still seen).
- Integer overflows leading to undersized allocations.
- Use-after-free in long-running daemons.
- Authentication bypasses via crafted URLs (e.g., directory traversal: `/../../etc/passwd`).

Exploitation is often easier than on desktop Linux because IoT devices typically lack many of the mitigations modern desktops have:

- ASLR is often disabled or partial – many old kernels don't enable it; many vendors disable it for “performance” or simplicity.
- Stack canaries are often missing (`-fstack-protector` not used at build time).
- DEP / NX (no-execute on data pages) is sometimes off.

- RELRO (read-only relocations) is rarely enabled.
- Full PIE (position-independent executables) is rare; binaries have predictable addresses.

A watchdog that restarts the daemon on crash actually helps the attacker: with a fixed binary address, you can brute-force ROP chains by attempting many crashes, each with a different guess. ASLR would defeat this; without ASLR, it just takes time.

2.5.4 Outdated kernels and userspace

Many IoT devices ship with kernels from 4–10 years ago. Public CVEs that were patched on desktop Linux long ago remain exploitable on these devices:

- DirtyCOW (CVE-2016-5195) – still exploitable on devices with kernels older than 4.8 with the relevant patches.
- Various netfilter / iptables CVEs.
- TCP stack vulnerabilities.
- Driver bugs.

Userspace components are similarly outdated. Dropbear (a tiny SSH server), uhttpd, OpenSSL versions, BusyBox versions – all can be looked up against public CVE databases. A device's CVE exposure can be enumerated just by extracting the firmware, listing the versions of binaries (`strings | grep -i version`), and matching against CVE databases.

2.5.5 The supply-chain problem

Many vendors don't actually write their firmware. They license a reference design from the SoC vendor (Realtek, MediaTek, Broadcom, etc.) and lightly customize it. Vulnerabilities in the reference design propagate to dozens of products across multiple brands.

Notable examples:

- The 2022 “Realtek SDK” family of vulnerabilities affected millions of devices across many brands – because all the brands shipped variations of the same Realtek-provided code.
- Similar issues have repeatedly affected MediaTek-based devices, Broadcom-based devices, and others.

Tracking which devices contain which underlying software is hard. The SBOM (Software Bill of Materials) movement – requiring vendors to disclose what's in their products – is trying to fix this. Tools like EMBA can generate SBOMs from firmware images. Adoption is slow.

Further reading

- Costin et al. 2014 – empirical attack surface paper.
- Antonakakis et al., “Understanding the Mirai Botnet” (USENIX Security 2017) – Section 2 on the IoT attack surface.
- Shodan and Censys research blogs – regular IoT exposure statistics.
- [CVE Details by vendor](#) – search “TP-Link”, “D-Link”, “Netgear”.

2.6 The Firmware Emulation Problem

2.6.1 Why emulate?

To dynamically analyze firmware – run it, observe it, fuzz it, watch malware operate on it – you have two basic options:

Run it on real hardware. Buy the device, connect a hardware debugger (typically via UART for console access, sometimes JTAG for deeper control), modify the firmware to be debuggable, instrument it as needed. Works, but doesn't scale: it's expensive (a copy of every device you want to study), slow (physically connecting and reflashing each device), and limited (you analyze what you have hardware for).

Emulate it. Run the firmware in QEMU on a regular workstation. Cheap, scalable, parallelizable. But emulation requires faking all the hardware the firmware expects to find, which is hard.

For academic research and large-scale analysis, emulation is usually the practical choice. For deep analysis of a specific device, hardware is sometimes necessary. The two complement each other.

2.6.2 Why emulation is hard

When a firmware image boots, it expects to find a specific universe:

- A specific CPU implementing a specific instruction set with specific features (FPU? NEON? Specific cache configuration?).
- Specific peripherals at specific memory-mapped addresses: a UART at 0x10000000, a GPIO controller at 0x10010000, a NIC at 0x20000000, sensors, radios, USB controllers, etc.
- NVRAM containing reasonable default values for thousands of configuration parameters.
- Specific kernel modules expected to be present for things like cryptographic acceleration, wireless drivers, vendor-specific functionality.
- Specific timing properties: “the watchdog must be kicked every 30 seconds or we reset”.
- Sometimes secure-boot signatures matching specific vendor keys.

QEMU can fake the CPU exactly. The rest is up to the emulation framework. When the framework guesses wrong – and it often does – the firmware fails in various ways:

- Kernel panic during early boot when it can't find an expected peripheral.
- Boot hang waiting for a hardware event that never comes.
- Successful boot but with no network interface, so you can't interact with the device.
- Successful boot but the web admin daemon won't start because its configuration came from NVRAM that's now empty.
- Successful boot, daemons start, but they immediately enter error states when they try to access hardware.

This collection of issues is the “firmware rehosting problem”. It's an active research area.

2.6.3 Two flavors of emulation

User-mode emulation. `qemu-arm-static` and friends. Run a single binary under QEMU, with QEMU translating syscalls on the fly to the host kernel. Useful for:

- Running a single ARM or MIPS binary on your x86 workstation.
- Fuzzing a specific component.
- Quick tests of binary behavior.

Limitations: you’re not booting a kernel, just running one binary against the host’s Linux. No init, no daemons, no networking stack from the firmware’s perspective. Many binaries fail because they expect a complete environment.

Full-system emulation. Tools like Firmadyne, FirmAE, and EMBA. Boot the whole firmware – bootloader, kernel, init, all daemons – in a QEMU virtual machine. The framework handles the hard parts: extracting the firmware, identifying its architecture, building or selecting a compatible kernel, faking NVRAM, mocking peripherals.

This is what you want for behavioral analysis. You can watch the device boot, interact with its web interface, run real exploits against it, observe its behavior under attack.

2.6.4 The history of firmware emulation tools

Firmadyne (2016). The first open-source framework for full-system IoT firmware emulation. Pioneered the approach: extract the firmware, identify its architecture, build a kernel, run with QEMU. Achieved approximately 16% success rate on a corpus of 1,124 router and IP camera firmwares – “success” meaning at least one network service reachable in the emulated environment. Largely unmaintained today but historically important.

FirmAE (2020). Improved on Firmadyne with the insight that most emulation failures had *common* causes that could be fixed with simple heuristics. The authors called this “arbitrated emulation” – when the firmware tries something that wouldn’t work (read missing NVRAM, talk to a missing peripheral), arbitrate by providing a plausible default. Achieved 79% on the same kind of corpus. Now lightly maintained.

EMBA (active development). Currently the state-of-the-art open-source framework. Performs both firmware-security-analysis (SBOM generation, CVE matching, weak-credential checks, etc.) and system emulation. Recent EMBA releases report:

- 95% success on the FirmAE corpus.
- 87% on a 2020 corpus from Fraunhofer FKIE’s Home Router Security Report.
- 77% on a 2024 corpus of more recent firmware.

For practical work in 2025+, EMBA is the right starting point.

2.6.5 What’s lost in emulation

Even when emulation succeeds, some things are inevitably different from real hardware:

- **Timing.** Emulated I/O is much slower than real I/O, which can change behavior in timing-sensitive code – TCP retry timeouts, watchdog intervals, race conditions.
- **Peripheral fidelity.** Sensors and radios are mocked with stubs that return canned values. Tests of features that depend on specific sensor values won’t work.

- **Performance counters.** CPU-specific features may not match.
- **Kernel modules.** Drivers for hardware that doesn't exist in emulation can't be loaded.
- **Boot timing.** The init order may differ if some daemons can't start.

For behavioral analysis of malware specifically, these limitations matter less than they would for vulnerability hunting or performance research. Malware mostly does syscalls and network operations, both of which emulate well. The bulk of malware behavioral signals are preserved in emulation.

Further reading

- Chen, Egele, Woo, Brumley, “Towards Automated Dynamic Analysis for Linux-based Embedded Firmware” (NDSS 2016) – the Firmadyne paper.
- Kim et al., “FirmAE: Towards Large-Scale Emulation of IoT Firmware for Dynamic Analysis” (ACSAC 2020) – Section 3 (failure analysis) is the most educational part.
- Wright et al., “Challenges in Firmware Re-Hosting, Emulation, and Analysis” (ACM Computing Surveys 2021) – survey for breadth.
- Muench et al., “What You Corrupt Is Not What You Crash” (NDSS 2018) – why dynamic findings on embedded systems are tricky.
- [EMBA wiki](#) – practical setup and usage.

2.7 The Current Tooling Landscape

2.7.1 Extraction

binwalk. The classic firmware extraction tool. Scans a binary blob for known signatures (filesystem magic numbers, compression headers, ELF binaries, vendor format headers) and reports or extracts what it finds. The `-e` flag extracts; `-Me` recursively extracts (useful for nested archives).

Workflow:

```
binwalk firmware.bin           # list what's inside
binwalk -e firmware.bin        # extract everything found
cd _firmware.bin.extracted/    # browse the result
ls -la                         # often: a SquashFS, some
    configs
unsquashfs squashfs-root.bin    # extract the rootfs
cd squashfs-root/              # now you have a real
    filesystem
```

Limitations: signature-based, so unknown or custom formats are missed. Sometimes overly heuristic – carves bytes that aren't really filesystem images. Not great with encrypted or signed firmware.

unblob. A newer alternative from OneKey (formerly IoT Inspector). More reliable extraction for many formats, including some that binwalk struggles with. Better at recursive extraction. Worth trying when binwalk has issues.

2.7.2 Reverse engineering

Ghidra. The NSA-released disassembler/decompiler. Free, open-source, mature. Supports all the architectures you'll encounter in IoT (x86, ARM, MIPS, PowerPC, SuperH, RISC-V,

and many more). Includes a decompiler that produces readable C-like pseudocode from binary, which is enormously useful.

Typical workflow:

1. Open a binary in Ghidra; it asks about the architecture (usually auto-detected correctly).
2. Let it auto-analyze (a few minutes for a large binary).
3. Navigate to `main()`, or find the entry point if there's no `main`.
4. Read the decompiled C output. Navigate function calls. Cross-reference strings.
5. Use the Symbol Tree to find imports (libc functions, syscall wrappers).

IDA Pro and Hex-Rays. The commercial standard. More polished UI than Ghidra; the Hex-Rays decompiler is excellent. Expensive (\$3,000+ for the full version). Most professional malware researchers use IDA; Ghidra is catching up rapidly.

Binary Ninja. Another commercial option, well-regarded for its clean API and Python integration.

radare2 / Cutter. Open-source, command-line-first (radare2) with a GUI (Cutter). Steep learning curve; powerful for those who master it.

For most IoT work, Ghidra plus occasional `objdump`, `readelf`, `strings`, `file` from the command line covers everything.

2.7.3 Dynamic analysis frameworks

strace, ltrace. The classic tracing tools (covered in Track 1).

eBPF-based tools. The modern stack (covered in Track 1). Specific behavioral monitoring tools built on eBPF:

- **Tracee** (Aqua Security) – runtime security tool with rich detection rules. Open source. Reading its signatures is excellent education on production behavioral detection.
- **Falco** (CNCF) – container-focused but works on any Linux. Rules express behavioral patterns in a high-level DSL.
- **Tetragon** (Cilium) – enforcement-capable eBPF security monitor.

Cuckoo Sandbox / CAPE Sandbox. Automated dynamic malware analysis. Submit a sample, the sandbox runs it in a VM and produces a behavior report. Cuckoo is the classic open-source tool; CAPE is a modern fork actively maintained. Both originated for Windows malware but have Linux support.

Drakvuf. VM-introspection-based malware analysis. Watches a guest VM from outside, so the malware can't easily tell it's being watched. Stealthy but harder to set up.

2.7.4 Where to get firmware samples

For analysis work, legally obtainable firmware:

- Vendor download pages (TP-Link, Netgear, D-Link, Asus all post firmware updates).

- OpenWrt downloads (downloads.openwrt.org) – hundreds of devices.
- Test corpora from research papers – the FirmAE and EMBA papers have public corpora, often on Zenodo.

For malware samples (use caution – handle in isolated environments):

- **MalwareBazaar** (bazaar.abuse.ch) – public ELF/Linux samples.
- **VirusTotal** – has academic / researcher access programs.
- **vx-underground** – community archive of samples and papers.
- **Mirai source code** – on GitHub in multiple mirrors. Searching “mirai-source-code” finds them. The source itself is far more educational than a compiled binary.

Watch out

Real malware samples should always be handled in isolated environments: a dedicated VM with no network access (or a snapshot you can roll back), or air-gapped hardware. Do not download samples to your daily-driver machine.

For learning, the Mirai *source code* is more educational than the binary. Reading well-commented C is faster than reverse-engineering a stripped binary, and you don’t need any isolation precautions to read source.

Further reading

- [binwalk GitHub](#).
- [unblob GitHub](#) – the newer alternative.
- [EMBA GitHub](#) – the active emulation/analysis project.
- [Ghidra documentation](#) – official “GhidraClass” tutorial exists.
- [HackTricks IoT pentesting section](#) – practical reference.

2.8 Track 2 Self-Check Questions

1. Why is the same C program producing different syscall numbers when traced on x86-64 vs ARM, and what does this imply for cross-architecture analysis tools?
2. What is NVRAM in a router context, and why does its absence break naive firmware emulation? How do FirmAE and EMBA address this?
3. Walk through how Mirai infects a vulnerable router at the protocol level – what ports, what credentials, what happens after login.
4. Why is BusyBox a problem for any naive “which binary is running” detection? How would you accurately identify the running BusyBox applet?
5. When would you use user-mode QEMU (`qemu-arm-static`) vs full-system emulation (EMBA)? What are the trade-offs?
6. Why are stack canaries and ASLR less common on IoT devices than on desktop Linux? What does this mean for exploit reliability?
7. Explain the supply-chain problem in IoT security with a specific example (e.g., the Realtek SDK vulnerabilities).
8. What’s the difference between SquashFS and JFFS2? Which is more commonly the root filesystem on IoT devices?

9. How does a CGI command-injection vulnerability typically arise? Why are these so common in IoT admin interfaces?
10. What information can you extract from an unknown firmware blob using `binwalk`, and where do you go next once you have the roots?

3 Malware Analysis as a Discipline

Malware analysis is a mature field with its own vocabulary, methodologies, and culture. To do meaningful work in it – whether detection, attribution, or reverse engineering – you need to understand the conceptual framework the field uses, the categories of techniques on both sides (attack and defense), and the specific history of the IoT malware landscape. This track is the densest in terms of background knowledge required.

3.1 The Taxonomy of Malware Analysis

3.1.1 Static vs dynamic vs hybrid

The fundamental split in malware analysis is between approaches that examine the binary without running it (static) and approaches that observe its behavior at runtime (dynamic). Each has strengths and weaknesses, and serious analysis usually combines them.

Static analysis. Examining a binary without executing it. The analyst looks at the bytes themselves: the disassembly, the strings embedded in the binary, the imported and exported symbols, the section structure, control flow graphs, data flow analysis.

Tools: `file`, `strings`, `readelf`, `objdump`, `nm`, plus reverse-engineering platforms like Ghidra, IDA Pro, Binary Ninja, radare2.

Advantages:

- **Safe.** You're not running potentially malicious code. The sample sits on disk; you read it.
- **Fast.** No sandbox setup, no waiting for the sample to act. Static features can be extracted in seconds.
- **Sees all code paths.** Even code paths that don't execute under any particular input are present in the binary and visible to static analysis. A dynamic trace shows what *did* happen; static analysis can reason about what *could* happen.
- **Scales.** You can statically analyze millions of samples cheaply. This is how antivirus signature databases get built.
- **Stable.** Static features don't depend on the analysis environment.

Limitations:

- **Packed or obfuscated code looks like noise.** If the malware decrypts itself at runtime, the static view shows only the decryptor and an encrypted blob – not the actual malicious code.
- **Polymorphism and metamorphism defeat simple signatures.** Different infections look different on disk even though they behave identically.
- **Runtime values are invisible.** You can't see which key was generated, which C2 server was contacted, which configuration was downloaded.
- **Behavior requires inference.** You see code structure but have to reason about what it does. Symbolic execution can help but is expensive.

Dynamic analysis. Running the binary in a controlled environment and observing what it does. The analyst watches as the sample executes: which syscalls it makes, which files it touches, which network connections it opens, which other processes it spawns, what data it writes to memory.

Tools: **strace**, sandboxes (Cuckoo, CAPE, Drakvuf), eBPF-based monitors, network captures (**tcpdump**, Wireshark), debuggers (gdb, x64dbg on Windows).

Advantages:

- **Sees actual behavior.** Including unpacked code that exists only in memory after the dropper runs.
- **Reveals runtime values.** C2 IPs, encryption keys, dropped files, network protocols are all visible.
- **Works on packed samples.** Once the malware unpacks itself in memory, you can dump and analyze the unpacked code.
- **Produces the kind of signal behavioral detectors use.** If you want to build a behavioral classifier, dynamic features are what you need.

Limitations:

- **Slow.** Minutes per sample, sometimes longer for samples that sleep or wait for triggers.
- **Sees only executed code paths.** If the sample's malicious behavior is triggered by specific conditions (a particular date, a C2 command, presence of certain files), and those conditions aren't met, the dynamic run reveals nothing useful.
- **Defeated by anti-analysis.** Malware that detects sandboxes will exit or behave benignly during analysis.
- **Requires careful isolation.** You're running real malware. Any sandbox failure could let it spread.

Hybrid analysis. Combines both. Static analysis identifies suspicious code paths and unpacks samples that decrypt themselves; dynamic analysis verifies behavior and captures runtime values. Symbolic execution (in tools like angr or KLEE) tries to reason about all possible execution paths statically, often using concrete values from limited dynamic runs to handle paths that are hard to reason about purely symbolically. Concolic execution (**concrete + symbolic**) is one specific hybrid technique.

Behavioral analysis. A subset of dynamic analysis that focuses on *what the program does to its environment* rather than *what it computes internally*. The features are external observables: syscalls, network packets, filesystem operations, process operations. Detection systems built on behavioral analysis abstract over implementation details – the same malicious behavior, implemented in different ways, produces similar behavioral signatures even if the bytecode differs.

Key idea

Behavioral analysis is at the right level of abstraction for IoT malware. The architectural diversity of IoT defeats instruction-level signatures – the same logic compiled for MIPS vs ARM looks completely different at the bytecode level. But *behaviors* (open Telnet sockets, scan IPs, attempt logins, kill competing processes) are architecture-independent. This is the central motivation for behavioral detection in this domain.

3.1.2 Where syscall-based detection fits

Within behavioral analysis, syscall-based detection is one approach among several. Alternatives include:

Network-based behavioral detection. Analyze packet captures. Tools like Zeek (formerly Bro) and Suricata produce structured logs of network behavior. Used heavily for botnet detection at the ISP level.

Filesystem-based detection. Watch for suspicious file writes, executable drops, modifications to startup scripts. Useful for persistence detection. Tools like `inotify` and `fanotify` provide the kernel-level interfaces.

Memory-based detection. Inspect process memory at intervals or on suspicious events. Look for injected code, unpacked malware, exposed credentials.

API-call-based detection. On Windows, hook the Win32 API rather than the syscall layer. The Linux equivalent would be hooking libc functions (via `LD_PRELOAD` or `uprobes`). Higher-level, sometimes easier to interpret, but loses signals about libc-internal behavior.

Sequence-based detection. Treat the syscall sequence as a sequence and use sequence models (HMMs, RNNs, transformers). The Forrest 1996 paper started this lineage; modern incarnations continue.

Graph-based detection. Build a graph from the trace – nodes are processes, edges are syscalls or relationships. Apply graph neural networks or other structural methods.

Syscall-based detection has specific advantages:

- Architecture-independent at the name level (after normalizing numbers to names).
- Hard to avoid – any meaningful malicious action requires syscalls.
- Well-instrumented (eBPF, `ptrace`, etc.).
- Granular enough to distinguish subtle behaviors.
- Universal – applies to any Linux malware regardless of category (worm, RAT, miner, botnet).

And drawbacks:

- High volume – a busy process can produce thousands of syscalls per second.
- Some semantic context is lost – you see `connect`, not “connecting to a known malicious C2”.
- Vulnerable to mimicry attacks where malware deliberately produces benign-looking syscall patterns.
- Per-system normalization required for cross-platform analysis.

Further reading

- Sikorski & Honig, *Practical Malware Analysis*, Ch. 1–3 – methodology introduction. Windows-focused but universal.
- Andriesse, *Practical Binary Analysis*, Ch. 1, 5–6 – Linux-focused and modern.
- Lenny Zeltser’s free introductions to malware analysis (SANS author page).

3.2 The Classic Malware Analysis Pipeline

A professional malware analyst examining a new sample typically follows a workflow that has evolved over decades:

3.2.1 Triage

Quick checks to decide what kind of sample you're dealing with and whether deeper analysis is justified.

- **file sample** – identifies the file type. ELF? PE (Windows)? Mach-O (macOS)? What architecture? Statically or dynamically linked? Stripped?
- Compute hashes (**sha256sum**, **md5sum**). Look them up in VirusTotal, MalwareBazaar. Is this a known sample? What family? How long has it been around?
- **strings -a sample | head -50** – the strings often give away the family immediately. “ELF binary containing the strings BRUTEFORCE_CONNECT and KILLER_DONE and Telnet port numbers” – that's Mirai.
- Check for packers. UPX is detected easily (**strings** shows “UPX!”). Custom packers require more work.
- Run **readelf -d** – which libraries does it link against? Statically linked is suspicious (malware often static-links to avoid library version issues on target devices).

Triage typically takes a few minutes per sample and decides the rest of the workflow.

3.2.2 Unpacking

If the sample is packed, you can't usefully analyze its code without unpacking it first:

- **UPX** – the most common public packer. Trivially unpacked with **upx -d sample**.
- **Custom packers** – specific to each malware family. Two main techniques to handle them:
 1. **Static unpacking** – reverse-engineer the unpacking algorithm and implement it as a script. Specific to each packer.
 2. **Dynamic unpacking** – run the sample in a debugger, wait for it to unpack itself in memory, dump the unpacked code from memory. Universal but requires running the malware.
- **No packing** – many IoT samples aren't packed at all. They're stripped (no debug symbols) and statically linked, but the actual code is in plain form. This is usual for Mirai-class malware where targets are too resource-constrained for elaborate runtime decryption.

3.2.3 Static analysis pass

Open the unpacked binary in Ghidra or IDA. Look for:

Imports. Which libc functions does it call? An ELF that imports **socket**, **connect**, **fork** but no GUI/UI functions is probably a daemon or scanner. An ELF that imports **ptrace** is probably doing anti-debugging. The combination of imports gives a quick taxonomy.

Strings. Beyond triage, careful string analysis reveals:

- Hardcoded C2 IP addresses and domain names.

- User-Agent strings used in HTTP requests.
- Default credential lists.
- Lists of process names to kill (the “competing malware” kill list).
- Lists of IP ranges to skip (Mirai’s exclusion list).
- Error messages and log strings – often left in even when stripped, and they uniquely fingerprint families.
- Encrypted strings (high entropy, often base64-like) suggesting the binary contains encrypted configuration that’s decrypted at runtime.

Function-level analysis. Find the entry point. Trace through the initialization code. Identify the main loop. Look for distinctive patterns:

- A loop calling `socket` + `connect` + `close` repeatedly with varying arguments – a scanner.
- A loop reading `/proc` and sending `kill` signals – a killer routine.
- Calls to `system()` with attacker-controlled-looking parameters – command execution.
- `mprotect` adding `PROT_EXEC` – code injection or self-modification.

Suspicious patterns. Hardcoded credentials, encryption keys (high-entropy constants), unusual call sequences, large jump tables (often byte-by-byte decryption loops), explicit anti-analysis (`ptrace` self-check, `/proc/self/status` reads).

3.2.4 Dynamic analysis pass

Run the sample in a sandbox. Collect:

- **Process tree.** What child processes does it spawn? Are they renamed via `prctl(PR_SET_NAME)`?
- **Filesystem activity.** What does it create, modify, delete? Where? Persistent or temporary?
- **Network activity.** pcap of all packets. DNS queries. TCP/UDP destinations. Protocols (HTTP, IRC, custom binary?).
- **Syscall trace.** The signal you’ll work with most for behavioral detection.
- **Memory dumps.** Snapshots of process memory at interesting points – often reveals unpacked payloads.
- **Loaded modules.** For samples that load kernel modules (rare in IoT, common in Windows rootkits).

The dynamic run is usually time-limited (5–10 minutes is typical) to keep the analysis pipeline moving. Samples that need more time or specific triggers may not reveal their full behavior in a single run.

3.2.5 Family clustering

Once you have features (static and dynamic), classify the sample into a family. The classical approaches:

Fuzzy hashing. ssdeep computes context-triggered piecewise hashes that are similar for similar files. Two builds of the same malware with minor variations will have similar ssdeep hashes; completely unrelated files will not.

telfhash. A TrendMicro tool specifically designed for IoT/Linux malware clustering. Computes a hash from the binary’s symbol table. Particularly effective for ELF malware because malware families often share symbol-table patterns even when the code differs.

Behavioral similarity. If two samples produce the same syscall sequences, contact the same C2 infrastructure, write the same files – they’re probably the same family. This is what behavioral classifiers learn.

ML-based clustering. Train embeddings on features (static or dynamic), cluster in the embedding space. More flexible than fixed hashes but requires labeled training data.

YARA rules. A domain-specific language for matching patterns in files (strings, byte sequences, structural properties). Widely used in industry for fast pattern matching at scale. A well-written YARA rule for Mirai might say “contains these specific strings AND has this many CGI-related strings AND is statically linked”. Family attribution via YARA is fast and explicit.

3.2.6 Reporting and IoCs

The final output of malware analysis is typically a set of *Indicators of Compromise* (IoCs): observable artifacts that defenders can search for to identify whether a system is compromised. IoCs include:

- File hashes (SHA-256 of the malware binary).
- IP addresses and domains contacted (C2 infrastructure).
- URLs visited.
- Mutex names, registry keys (Windows-specific).
- Filesystem artifacts – paths the malware writes to, characteristic file names.
- Network signatures: specific User-Agent strings, specific packet patterns, characteristic timing.
- YARA rules for matching variants.
- Behavioral signatures (Sigma rules, Tetragon policies, etc.).

IoCs are shared via threat intelligence feeds, vendor reports, MISP (Malware Information Sharing Platform), and academic publications.

Further reading

- Sikorski & Honig, Ch. 4–7 – triage, basic static, basic dynamic.
- TrendMicro’s telfhash blog post and the underlying paper.
- Cuckoo Sandbox documentation – read the architecture overview.
- [CAPE Sandbox](#) – modern fork of Cuckoo.
- [YARA documentation](#) – for pattern-matching at scale.

3.3 Static Analysis Features

These are features extracted from the binary without running it. Useful for fast, cheap classification at scale, and as inputs to ML models.

3.3.1 Strings

The simplest, most powerful static feature. `strings -a binary | sort -u` produces all printable strings of length ≥ 4 in the binary. (The default minimum is 4; pass `-n 6` for length 6 minimum, etc.) For IoT malware specifically, strings often reveal:

- C2 server IPs or domain names. Often hardcoded; sometimes only base64-encoded or XOR-obfuscated.
- User-Agent strings used in HTTP requests (these often fingerprint specific families).
- Default credential lists.
- Lists of process names to kill (the “competing malware” kill list).
- Lists of IP ranges to skip (Mirai’s exclusion list of US government and corporate ranges).
- Error messages and log strings – often left in even when stripped, and they uniquely fingerprint families.
- Function names or symbol names that didn’t get stripped.
- Build artifacts: compiler version strings, file paths from the build environment, timestamps.

For machine learning features, raw strings are too noisy (very high dimensional, mostly irrelevant). Better approaches:

- Extract specific structured information: IPs (regex), domains, URLs.
- Use string entropy as a feature: a binary with many high-entropy strings is probably packed or has encrypted data.
- Build a vocabulary of “interesting” strings (those appearing in known malware) and use presence/absence as binary features.

3.3.2 Imports and dynamic symbols

`readelf -d binary` shows the dynamic dependencies (which libraries this binary expects to find). `readelf -dyn-syms` or `nm -D` lists imported symbols (which specific functions it calls from those libraries).

For ELF malware:

- **Statically-linked binaries are common.** Malware often static-links to avoid version compatibility issues across target devices. `file` will say “statically linked”. Static linking strips out the dynamic symbol information; you have to identify functions by signature matching or other techniques.
- **Dynamically-linked malware** imports a characteristic small set of libc functions: `socket`, `connect`, `fork`, `exec*`, `ptrace`. The set of imports gives a quick taxonomy of what the malware needs.
- **Heavy stripping** is itself a feature. A real application doesn’t strip away every symbol. A heavily stripped binary often has analysis avoidance as a goal.

For ML features, the import set can be encoded as a bag-of-functions vector. “This binary imports `socket`? Yes/no. `connect`? Yes/no. ...”.

3.3.3 Opcode n-grams

Disassemble the binary and treat the sequence of CPU opcodes as a sequence. Build n-grams (e.g., 3-grams of opcodes: “`push`, `mov`, `call`”). Useful for:

- Family classification when malware samples share substantial code.
- Detecting packed sections (packed code has unusual opcode distributions, often biased toward XOR, ROL, and other simple operations).
- Identifying compiler artifacts.

Limitations:

- Completely defeated by recompilation for a different architecture.
- Sensitive to compiler optimization levels.
- Doesn’t capture semantic-level information.
- High dimensional (the opcode set is large, n-grams are larger).

For cross-architecture analysis, opcode n-grams are essentially useless. For within-architecture family clustering, they can be useful.

3.3.4 Control flow graph features

Build the function call graph (which functions call which) and the basic-block graph (within each function, the flow of execution). Extract features like:

- Number of functions.
- Average basic blocks per function.
- Maximum cyclomatic complexity.
- Presence of specific structural patterns (a function that loops calling `connect` – a scanner; a function that enumerates `/proc` entries – a process enumerator).
- Graph-theoretic features: clustering coefficient, longest path, betweenness centrality.

CFG features are more robust to some changes (variable renaming, instruction reordering) than opcode features, but more expensive to extract (requires correct disassembly, which can be hard for packed or obfuscated code).

3.3.5 ELF metadata

Things you can read directly from the ELF header and section table without disassembling:

- **Architecture and endianness.**
- **Statically vs dynamically linked.**
- **Stripped vs unstripped.**
- **Compiler / linker used** (often visible in `.note.gnu.build-id` or `.comment` sections).
- **Presence of unusual sections.** UPX adds sections named `UPX0`, `UPX1`, `UPX2`. Other packers have their own signatures.

- **Section permissions.** A writable+executable section is unusual and suspicious.
- **Entry point location.** If it's in an unusual segment (not the standard text segment), the binary may be packed or modified.
- **Symbol density.** How many symbols per byte of code? Very few symbols suggests stripping.

These are quick to extract and useful as preliminary features.

3.3.6 Function-level signatures

Several techniques for hashing or signaturing functions in a binary, useful for family attribution:

telfhash. Hashes the imported symbol table of an ELF binary. Two malware samples from the same family typically have very similar telfhashes because they share imports. Designed specifically for IoT/Linux malware clustering. Originally from TrendMicro.

TLSH (Trend Micro Locality Sensitive Hash). A more general fuzzy hash. Similar files have similar TLSH values.

ssdeep. Older context-triggered piecewise hashing. Similar in spirit to TLSH.

These are useful for clustering and family attribution but not for distinguishing malicious from benign (a benign binary has its own hash that doesn't carry malicious-vs-benign signal until you look it up in a database).

Further reading

- Andriesse, *Practical Binary Analysis*, Ch. 5–7 – static analysis of ELF.
- Saxe & Berlin, “Deep Neural Network Based Malware Detection Using Two Dimensional Binary Program Features” – early influential static-feature deep-learning paper.
- Wang et al., “A Novel Static Analysis Approach Using System Calls for Linux IoT Malware Detection” (Electronics 2024) – recent.

3.4 Dynamic Analysis Features

These are the features a behavioral classifier actually uses. Understanding what each looks like, how it's extracted, and what it captures is essential.

3.4.1 Syscall sequences and frequencies

The core behavioral signal. Run the binary, collect the sequence of syscalls it makes. The trace might look like (abbreviated):

```
execve("/tmp/x", ["/x"], 0x7ffd...) = 0
brk(NULL) = 0x55a3...
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1,
0) = 0x7f8...
arch_prctl(ARCH_SET_FS, 0x7f8...) = 0
set_tid_address(0x7f8...) = 4521
socket(AF_INET, SOCK_STREAM, IPPROTO_TCP) = 3
connect(3, {sa_family=AF_INET, sin_port=htons(23), sin_addr=...}, 16)
= -1 ECONNREFUSED
close(3) = 0
```

```
socket(AF_INET, SOCK_STREAM, IPPROTO_TCP) = 3
connect(3, {sa_family=AF_INET, sin_port=htons(23), sin_addr=...}, 16)
    = -1 ETIMEDOUT
close(3) = 0
... (thousands more)
```

This raw trace must be transformed into features a classifier can use. Several common representations:

Frequency vector. A vector of length $|S|$ (number of distinct syscalls in your alphabet), where entry i is the count of how many times syscall i appeared in the trace. Simple, scales easily, surprisingly effective.

A frequency vector for a typical Mirai bot’s first few seconds might be dominated by `socket`, `connect`, `close`, `rt_sigaction`; a benign shell-utility-running process is dominated by `openat`, `newfstatat`, `read`, `close`.

Limitation: ignores ordering. The trace “socket, connect, send, recv, close, socket, connect, send, recv, close” and “socket, socket, connect, connect, send, send, recv, recv, close, close” produce identical frequency vectors but mean very different things.

N-grams. Sequences of n consecutive syscalls, with their frequencies. 3-grams and 5-grams are typical. Captures local ordering, which frequency alone misses. A scanning loop produces 3-grams like (socket, connect, close), (connect, close, socket), (close, socket, connect) repeatedly.

N-gram features can grow combinatorially: if there are $|S|$ syscalls, there are potentially $|S|^n$ n-grams. In practice the realized n-grams are much sparser (most combinations never occur), but the feature vector can still be wide. Feature selection or dimensionality reduction is often needed.

Sliding window histograms. Counts of syscalls in fixed-size windows of N consecutive calls. Detects bursts of specific behavior – a window with 95% `connect` calls is very different from one with 5%.

Raw sequence for sequence models. Feed the syscall sequence directly to a sequence model (LSTM, GRU, transformer). Captures arbitrary-distance dependencies. Requires substantial training data and careful evaluation; often overkill.

Per-FD sequences. Track which syscalls happen on which file descriptor. Separates network activity from filesystem activity from IPC. Adds context to otherwise-ambiguous events.

Per-process trees. For multi-process malware (or any non-trivial workload), group syscalls by process and analyze each process’s trace separately, or build cross-process features (parent-child relationships, signal patterns).

3.4.2 Network behavior

Captured either separately (`tcpdump`, packet capture) or extracted from network-related syscalls (`socket`, `connect`, `sendto`, `recvfrom`, `accept`). Common features:

- Number of distinct destinations contacted (high = scanner; low = focused C2).
- Distribution of destination ports (a long tail = scanner; one port = C2).

- Connection success/failure ratio (high failure rate = scanning random IPs; low failure = known-good targets).
- Protocols used (TCP, UDP, ICMP, raw sockets).
- Packet sizes and timing patterns (regular small packets = heartbeat; bursty large packets = exfiltration; flat-out high-rate UDP = DDoS).
- DNS query patterns (DGA-style high-entropy domains, many TXT lookups, fast-flux behavior).

For Mirai's scanner: hundreds of `connect` attempts per second, mostly to port 23, with a very high failure rate (most random IPs don't have Telnet open or refuse the connection). This is extremely distinctive.

3.4.3 Filesystem activity

What does the binary read, write, modify, delete? Features:

- Files created in specific paths: `/tmp/`, `/var/run/`, `/dev/shm/`, `/etc/init.d/` (persistence).
- Modifications to startup scripts (persistence via `/etc/rc.local`, init scripts, systemd units, crontabs).
- Creation of executable files followed by `chmod +x`.
- Deletion of evidence – many droppers delete themselves immediately after running.
- Reads of system identification files (`/etc/hostname`, `/proc/cpuinfo`) early in execution – often environment fingerprinting.

For Mirai: almost no filesystem activity at all. Mirai is memory-resident – it writes nothing to disk by design. This very absence is itself a behavioral signature: a process with substantial CPU and network activity but no filesystem activity is unusual for legitimate software.

3.4.4 Process activity

The process tree and operations:

- Number of children spawned.
- Fork rate over time.
- Patterns of `execve` – which programs does this process invoke?
- Signals sent: `kill` to many distinct PIDs is the signature of a killer routine.
- Process renaming via `prctl(PR_SET_NAME)` – often to hide as a system process.
- Daemonization patterns: `fork` + `setsid` + parent exit.
- Reading from `/proc/<pid>/` for many distinct PIDs – enumerating other processes.

3.4.5 Memory patterns

- Heap growth rate (`brk` call frequency and size).
- `mmap` patterns – many large allocations is unusual.
- `mprotect` adding `PROT_EXEC` to previously non-executable pages – code injection or JIT.
- Reading from `/proc/self/maps` – self-inspection, often anti-debugging.

- Reading from `/proc/<other-pid>/mem` – process memory inspection, often associated with credential theft.

3.4.6 Combining feature modalities

In practice the strongest classifiers combine multiple feature types:

- Syscall frequency vector (captures overall “shape” of execution).
- Top-N syscall 3-grams (captures local sequence patterns).
- Network-level features (destinations, ports, timing).
- Process-tree features (fan-out, signal patterns).

The exact weighting and combination depend on the detection task and the deployment constraints.

Try this

Hands-on: write a small Python program that connects to many random IPs on port 23 with short timeouts (a synthetic Mirai-like scanner). Trace it with `strace -c` for syscall counts and `strace -e trace=network` for the network syscall sequence. Compare with `strace -c` on `curl` fetching a single URL. The frequency vectors are completely different shapes – this is the signal classifiers exploit.

Then trace your synthetic scanner with `bpfttrace` and compute n-gram frequencies in Python. Plot the top-20 3-grams. You’ll see a small number of n-grams dominating, reflecting the tight scan loop.

Further reading

- Canzanese, Mancoridis, Kam, “System Call-Based Detection of Malicious Processes” – modernizes Forrest with ML.
- Cozzi, Graziano, Fratantonio, Balzarotti, “Understanding Linux Malware” (IEEE S&P 2018) – the largest empirical study of Linux/IoT malware. Sections 4–5 cover dynamic features in detail.
- Cozzi’s talk presenting that paper at IEEE S&P 2018 – video on YouTube.

3.5 Evasion Techniques

Behavioral detection is in an arms race with malware authors who actively try to defeat it. Understanding the major evasion techniques tells you what your detectors need to be robust to, and gives you the conceptual vocabulary to read evasion-aware malware research.

3.5.1 Anti-debugging

Techniques specifically aimed at detecting debuggers (gdb, IDA Pro debugger, ptrace-based analysis tools).

ptrace self-check. The malware calls `ptrace(PTRACE_TRACEME, ...)` on itself. If successful, no debugger is currently attached (because only one tracer can be attached at a time). If it fails with `EPERM`, a debugger is already there. Mirai uses this.

TracerPid check. Read `/proc/self/status` and inspect the `TracerPid:` field. If non-zero, the process is being traced. This is the cheapest possible check – one file read.

Timing checks. Measure how long simple operations take. Under single-step debugging or heavy instrumentation, normal operations become much slower. `rdtsc` on x86 (or `clock_gettime` on any architecture) before and after a known operation, compared to a baseline, reveals tracing.

Breakpoint detection. Software breakpoints work by replacing the target instruction byte with INT3 (0xCC). Scanning your own code for 0xCC bytes that shouldn't be there reveals breakpoints.

Hardware breakpoint detection. The debug registers DR0–DR3 hold hardware breakpoint addresses. A process can read its own debug registers to see if any are set.

Parent process check. A malware sample can check its parent process name – if it's "gdb" or "strace" or similar, something is up.

Counter-techniques: eBPF doesn't appear in `TracerPid`, doesn't use `ptrace`, doesn't insert breakpoints. It defeats all the classic anti-debugging at once. This is one of eBPF's biggest advantages for security research.

3.5.2 Anti-VM

The malware tries to detect that it's running in a virtual machine (rather than on real hardware) and exits or behaves differently. Common techniques:

CPUID checks. On x86, the CPUID instruction returns CPU identification information. Hypervisors set a specific bit (the "hypervisor present" bit), and many leave identifying strings ("KVMKVMKVM", "VMwareVMware", "Microsoft Hv") in CPUID leaves. A simple CPUID read can reveal the hypervisor.

Hardware fingerprinting via `/proc` and `/sys`. Reading `/proc/cpuinfo`, `/proc/scsi/scsi`, `/sys/class/dmi/id/product_name`, `/sys/class/dmi/id/sys_vendor` reveals VM signatures ("QEMU", "VMware", "VirtualBox", "innotek GmbH").

MAC address ranges. Specific OUI (organizationally unique identifier) prefixes are assigned to VM vendors. 08:00:27 is VirtualBox, 00:0c:29 and 00:50:56 are VMware. Reading the MAC of `eth0` reveals VMs.

Timing-based VM detection. Some operations have characteristic timing in VMs vs bare metal (especially TLB flushes, certain interrupts).

Device file checks. `/dev/vbox*` indicates VirtualBox guest additions; specific virtio device files indicate KVM/QEMU; `/dev/vmci` indicates VMware.

Counter: a well-set-up analysis sandbox modifies all of these to look like real hardware. This is a significant engineering effort but doable. EMBA's emulation includes many of these mitigations. The Cozzi 2018 paper found that many Linux/IoT malware samples don't bother with sophisticated anti-VM, possibly because the sandbox ecosystem for Linux is less developed than for Windows.

3.5.3 Anti-sandbox

Less about detecting "a VM" specifically, more about detecting "an automated analysis environment". Often more sophisticated than anti-VM because real production environments *are* often VMs (cloud servers).

Sleep-then-check. The malware sleeps for a long time at startup (an hour, a day) and then checks if anything has changed. Many sandboxes time out before the sleep finishes. Counter: modern sandboxes detect long sleeps and “fast-forward” time by patching the sleep syscall.

Requiring user interaction. The malware waits for a mouse click, keyboard input, or specific user action. Defeats sandboxes that only run binaries with no UI interaction. Doesn’t apply to IoT malware much (IoT devices typically don’t have users in the usual sense).

Hostname or username checks. If the hostname is “sandbox”, “cuckoo”, “test”, or matches certain patterns – exit. Trivial to defeat by changing the hostname.

Network reachability checks. “If I can’t reach my C2 server, maybe I’m in an isolated sandbox – exit”. Defeats sandboxes that don’t allow outbound network. Counter: provide controlled outbound network with fake C2 servers (INetSim, FakeNet).

Artifact checks. Look for analysis tool processes (`strace`, `wireshark`, `gdb`, `tcpdump`), specific filesystem layouts, mounted analysis tools.

Wear-and-tear checks. Real machines have signs of use: long uptime, many users, many installed programs, browser history, recently used files. A pristine sandbox doesn’t. The Miramirkhani 2017 paper introduced this idea.

3.5.4 Packing and obfuscation

Packers. Compress or encrypt the malware body, prepend a stub that unpacks at runtime. The static view shows mostly the stub and the encrypted blob.

- **UPX** – the most common public packer. Trivially defeated.
- **Custom packers** – specific to each malware family. Harder to defeat statically; must run the malware to observe the unpacked state.

For IoT malware specifically, packing is less common than on Windows. Targets are resource-constrained, and statically-linked stripped binaries already provide reasonable analysis-resistance.

Control-flow obfuscation. Insert junk instructions, branches that always go one way, opaque predicates (conditions that always evaluate to a specific value but are hard to evaluate statically). Defeats simple disassembly. Aggressive control-flow flattening can make a function with 50 basic blocks look like a single dispatcher with 50 cases.

String encryption. C2 IPs, credentials, command names – the strings that most reveal a sample’s purpose – are encrypted in the binary and decrypted only when used. Defeats string-based static analysis but doesn’t help against dynamic analysis that captures memory after decryption.

API hashing. Instead of calling functions by name (`LoadLibrary("user32.dll")`, `GetProcAddress("Message")`), the malware computes hashes of function names and looks them up in loaded libraries by hash. Defeats import-table analysis. More common on Windows than Linux.

3.5.5 Polymorphism and metamorphism

Polymorphism. Each infection encrypts the malware body with a different key. The decryptor stub is constant; defenders sign the decryptor. The unpacked code is identical across infections, just encrypted differently on disk.

Metamorphism. Each infection rewrites the code itself: different instruction sequences that achieve the same effect, different register allocation, different control flow. Same behavior, different bytecode. Much harder to implement; rarely seen in practice.

Both polymorphism and metamorphism are defeated by behavioral analysis (the behavior is invariant) but troublesome for static analysis (especially signature-based).

3.5.6 Why IoT malware uses fewer evasion techniques

Empirically, IoT malware uses much less sophisticated evasion than typical Windows malware. The reasons are structural:

- **Target constraints.** Routers and cameras have limited CPU and memory. Elaborate runtime decryption is expensive.
- **Weak defender landscape.** You don't need to evade what isn't there. Most IoT devices have no AV or behavioral monitoring at all. Static stripping is already sufficient against most actual defenders.
- **Short useful lifetimes.** A device reboots and Mirai-class malware is gone – but the device gets reinfected in seconds because it's still vulnerable. Persistence-oriented evasion has lower payoff than for Windows malware.
- **Author sophistication.** Many IoT botnets are derived from leaked source code by relatively unsophisticated operators ("script kiddies"). The original Mirai author was technically capable; most subsequent variant authors are not.

This is changing. Mozi and more recent families show increasing sophistication. Expect future IoT malware to be harder to analyze.

Further reading

- Sikorski & Honig, Ch. 15–18 – anti-disassembly, anti-debugging, anti-VM. Excellent coverage.
- [Check Point “Anti-Analysis Techniques” wiki](#) – comprehensive catalog.
- Miramirakhani, Appini, Nikiforakis, Polychronakis, “Spotless Sandboxes: Evading Malware Analysis Systems Using Wear-and-Tear Artifacts” (IEEE S&P 2017).

3.6 The IoT Malware Landscape

To understand the field, you need to know its history. The IoT malware landscape has been shaped by a small number of pivotal families.

3.6.1 The pre-Mirai world

Before 2016, IoT malware existed but was a curiosity rather than a major threat category. The main pre-Mirai families:

Linux.Hydra (2008). Sometimes called the first IoT botnet. Targeted MIPS routers via SSH brute force. Small scale. Demonstrated the concept.

Psybot (2009). A worm targeting home routers running Linux. Brute-forced credentials, recruited routers into a botnet for DDoS. One of the first to show that IoT botnets at scale were possible.

Linux.Aidra (2012). Targeted CCTV DVRs via default credentials. A precursor to the Mirai approach.

Carna botnet (2012). Built by an anonymous researcher who scanned the entire IPv4 internet from compromised IoT devices (default-credentialed routers). Not maliciously used – the researcher published an internet census instead – but demonstrated that the underlying access existed at massive scale. “If I could do this for a research project, anyone could do this for a botnet.”

Gafgyt / Bashlite / Lizkebab (2014). A direct precursor to Mirai. Telnet brute force, DDoS-focused. Source code leaked in 2015, providing the template that Mirai’s author would later improve on. Gafgyt-derived malware continues to exist in parallel with Mirai-derived malware to this day.

3.6.2 Mirai (2016): the watershed

Released as a fully working open-source IoT botnet by an actor calling themselves “Anna-Senpai” on Hackforums in late September 2016. The source code release essentially democratized IoT botnet creation: anyone could now spin up their own variant.

Mirai was responsible for the largest DDoS attacks on record at the time:

- **Against KrebsOnSecurity**, September 2016, 620 Gbps.
- **Against OVH**, September 2016, approximately 1 Tbps (1000 Gbps).
- **Against Dyn DNS**, October 2016, taking down large portions of the US internet for several hours (Twitter, Reddit, Spotify, Airbnb, Netflix, GitHub all affected).

Mirai’s architecture:

- **Bot** – runs on infected devices. Scans for new victims, executes DDoS commands.
- **Scanner** – a component within each bot that randomly generates target IPs and attempts Telnet brute-force login.
- **Reporter** – on each bot, sends successful credential pairs back to a central server.
- **Loader** – a separate server that pulls reports and infects newly-discovered vulnerable devices.
- **CnC (Command and Control)** – the operator’s interface for issuing attack commands to the bot network.

The key design decisions:

- **Memory-resident only.** Mirai writes nothing to disk. A reboot kills the infection.
- **Aggressive killer routine.** Kills competing malware on the device, binds to telnet/SSH ports to prevent re-infection.
- **Hardcoded credential list.** About 60 username/password pairs targeting specific device classes.

- **IP exclusion list.** Avoid scanning US Postal Service, DoD, GE, HP, IANA-reserved ranges – likely to avoid attracting law enforcement attention.
- **Multiple architecture support.** The loader determines the target’s architecture and delivers the right binary.

The Antonakakis et al. 2017 paper (USENIX Security) is the definitive academic analysis of Mirai and should be required reading.

3.6.3 Mirai variants and descendants

After the source code leak, dozens of variants appeared:

- **Owari** – minor variant, similar credential list, slightly modified attack types.
- **Satori** – added exploit-based propagation. Used CVE-2014-8361 (Realtek SDK) and CVE-2017-17215 (Huawei HG532) to infect devices that wouldn’t fall to credential brute-forcing. Substantially more dangerous than original Mirai.
- **Okiru / Akiru** – targeted ARC processors specifically. Unusual architectural targeting.
- **Masuta / PureMasuta** – added more exploits.
- **JenX** – targeted Grand Theft Auto private servers (creative reuse of the same DDoS code).
- **Wicked** – expanded the exploit list further (Netgear, Western Digital, others).
- **Hakai, Yowai, Sora** – minor variants with marginal improvements.

The common thread: the Mirai source code is the template; variants add new credentials, new exploits, sometimes new DDoS attack types. The “Tangled Genealogy of IoT Malware” paper by Cozzi et al. tracks family relationships across 93,000+ samples and shows just how interconnected this family tree is.

3.6.4 Gafgyt and parallel evolution

Gafgyt/Bashlite predated Mirai and continues to exist alongside it. Its source code is also public. Similarities: Telnet brute-force, DDoS focus. Differences: longer codebase, more features, broader exploitation, generally seen as a more “mature” codebase by users.

The Gafgyt family and the Mirai family have cross-pollinated extensively. Modern IoT botnets often combine techniques from both.

3.6.5 Hajime (2016)

A “vigilante” botnet. Similar infection tactics to Mirai but with an unusual twist: instead of using infected devices for attacks, Hajime patched them against future infection by closing the vulnerable ports and displayed a message claiming benevolent intent. Demonstrated that the same infection capabilities could be used for different goals. Memory-resident like Mirai. The authors were never identified.

3.6.6 Hide-and-Seek (2018)

The first IoT botnet to use a true peer-to-peer architecture rather than centralized command-and-control. Each bot can act as a server for other bots. Much harder to take down because there’s no single C2 to seize. Demonstrated that IoT botnets were becoming architecturally more sophisticated.

3.6.7 Mozi (2019)

The natural successor to Mirai. Peer-to-peer architecture based on the BitTorrent DHT (Distributed Hash Table) protocol – a clever reuse of an existing, hard-to-block protocol. More sophisticated than original Mirai. Heavily targeted Chinese-vendor devices. As of late 2021 reports, responsible for the majority of IoT malware traffic measured in some honeypots.

Mozi was reportedly disabled in mid-2021 (the original operators may have been arrested), but variants continue to circulate.

3.6.8 IoTroop / Reaper (2017)

Notable for shifting decisively from credential-based to exploit-based propagation. Used about 9 known vulnerabilities in various router/camera models. Demonstrated that the IoT-malware game was no longer just about default passwords – well-maintained exploit chains were now part of the standard toolkit.

3.6.9 More recent families (2022–2025)

BotenaGo. A Mirai-like botnet written in Go. Unusual for IoT malware (which is typically C). Statically-linked Go binaries are larger but easier to port across architectures without recompilation issues.

EnemyBot. A 2022 family combining exploits from Mirai, Gafgyt, and original ones. Active C2 infrastructure for an extended period.

Mirai variants targeting Log4Shell (CVE-2021-44228). Showed Mirai-family adaptability to non-IoT-specific vulnerabilities. Some variants exploited Log4j to infect any vulnerable Java-running device.

Dark Frost. 2023 family targeting gaming companies for DDoS.

InfectedSlurs. 2023 family exploiting zero-days in specific router models.

The general trends:

- More exploits, fewer brute-force-only families.
- More sophistication (P2P, multiple persistence mechanisms).
- More language diversity (Go, Rust appearing alongside C).
- Some persistence techniques beyond pure memory residence.
- More targeted attacks alongside the indiscriminate scanning approach.

Further reading

- Antonakakis et al., “Understanding the Mirai Botnet” (USENIX Security 2017) – definitive on Mirai.
- Cozzi, Vervier, Dell’Amico, Shen, Bilge, Balzarotti, “The Tangled Genealogy of IoT Malware” – traces family relationships across 93k+ samples.
- Vendor threat reports (TrendMicro, Kaspersky, Akamai, Radware, 360 Netlab) – search “Mozi botnet analysis”, “Mirai variants 2024”.
- Krebs on Security, “Who is Anna-Senpai, the Mirai Worm Author?” – the human story behind the leak.

3.7 Mirai Specifically – What to Know Cold

You should be able to explain Mirai in detail, because it's the reference point for every IoT malware discussion. This section is the conceptual map; the actual deep understanding comes from reading the source code.

3.7.1 The infection lifecycle, end to end

Stage 1: Scanner. The bot, already running on a previously-infected device, generates random IPs (with the exclusion list applied – US Postal Service, DoD, GE, HP, IANA-reserved). For each generated IP, it attempts to connect to TCP port 23 (Telnet), and on some variants port 2323 as well. On successful connection, it tries each of approximately 60 hardcoded username/password pairs in sequence.

The success rate is remarkably high. The credential list was tuned to specific device classes, and many of those devices have unchangeable defaults. A few percent of randomly-scanned IPs respond and accept one of the credentials. At the scale of millions of bots scanning continuously, this finds new victims at a rate of tens of thousands per day.

Stage 2: Report. On successful login, the scanner doesn't immediately infect the victim. Instead, it sends the victim's IP, port, username, and password back to a central "report server" over a simple binary protocol. This separation is important: it lets a single loader handle infections from many scanners.

Stage 3: Loader. A separate loader process pulls entries from the report server and processes them one at a time:

1. Reconnect to the victim's Telnet using the reported credentials.
2. Determine the victim's CPU architecture. This is done by trying `uname`, by examining filesystem clues, and by trial-and-error binary downloads.
3. Try multiple methods to deliver the bot binary to the victim:
 - `wget` – if the victim has it. Quick.
 - `tftp` – if `wget` isn't available but `tftp` is.
 - **The echo loader** – if neither is available, use shell echo commands to transfer the binary one byte at a time, encoded as printable characters. This is incredible to read in the source code. It's slow but works on the most minimal busybox installations.
4. Execute the binary on the victim.
5. Disconnect.

Stage 4: New bot startup. The newly-infected device's bot binary starts up:

1. Calls `ptrace(PTRACE_TRACEME)` on itself as an anti-debugging check.
2. Changes its process name via `prctl(PR_SET_NAME)` to something innocuous-looking.
3. Forks into the background and exits the parent (daemonization).
4. Binds to specific ports (typically 23 and sometimes 22, 80) to prevent other malware from using them and to make the bot harder to forcibly remove (the bound port can't be re-bound by another process).
5. Starts the killer routine in a background thread.

6. Starts the scanner routine in another background thread (looping back to Stage 1).
7. Connects to the C2 server and waits for attack commands.

Stage 5: DDoS on command. When the operator issues a DDoS command via the CnC, all connected bots execute the attack in parallel. The bots themselves do the packet generation; the C2 server just sends a tiny command (target IP, port, attack type, duration). One command can launch a Tbps-scale attack.

3.7.2 The killer routine

This is the part worth studying carefully because it's an unusually clean behavioral signature.

The killer:

1. Scans `/proc` for all running processes by reading directory entries.
2. For each process, reads its `cmdline`, `exe` symlink target, and (sometimes) memory.
3. Matches against a hardcoded kill list:
 - Names of competing IoT malware (Anime, other Mirai variants).
 - Processes using ports 22, 23, or 80 (telnet, ssh, web).
 - Processes whose executables live in suspicious paths (`/var/run/`, `/tmp/`, `/dev/`).
4. Sends `SIGKILL` to matched processes.
5. Binds to ports 22, 23, 80 itself to prevent re-binding by other processes.
6. Periodically re-runs to catch new processes.

Key idea

The killer routine is a fantastic behavioral signal. A process that:

- Opens `/proc` and enumerates many PID directories
- Reads each PID's `cmdline` and `exe`
- Sends `SIGKILL` to many distinct PIDs
- Then binds to port 23

is almost certainly Mirai-class malware. No legitimate process does this combination. Your behavioral detector should pick up this combination of behaviors as a single signature. Even better, the killer routine is hard to evade without making the malware much less effective – the entire point of killing competitors is to monopolize the device. Removing the killer means losing turf to other malware.

3.7.3 Memory residence

Mirai writes nothing to persistent storage. The loader downloads the bot binary, often to `/tmp/` or `/dev/shm/` (both typically tmpfs on IoT devices, vanishing on reboot), but the running bot never modifies any persistent file.

Consequences:

- A reboot wipes Mirai. The device is uninfected until the next scanner finds it – which typically takes seconds for any device on a public IP.
- Forensic analysis on a powered-off device finds nothing.

- Filesystem-based antivirus scans find nothing.
- The infection is entirely dependent on the ecosystem – the central report server and loader infrastructure must keep operating to reinfect after every reboot.

For behavioral detection, memory residence is itself useful as a feature: a process with substantial network and CPU activity but no filesystem activity is unusual for legitimate software.

3.7.4 The IP exclusion list

In `bot/scanner.c`, the source code has commented entries like:

```
// 127.0.0.0/8      - Loopback
// 0.0.0.0/8        - Invalid address space
// 3.0.0.0/8        - General Electric Company
// 15.0.0.0/7       - Hewlett-Packard Company
// 56.0.0.0/8       - US Postal Service
// 6.0.0.0/7        - Department of Defense
// 11.0.0.0/8       - Department of Defense
// 21.0.0.0/8       - Department of Defense
// 22.0.0.0/8       - Department of Defense
// ...
```

The exclusions for DoD/USPS/GE/HP are likely an attempt to avoid attacking targets that would attract serious law-enforcement attention. They’re also a cultural artifact – a glimpse into how the author thought about risk and which lines they thought they shouldn’t cross. The comments in the source are worth reading just for this human dimension.

3.7.5 The DDoS attack types

Mirai implements several attack primitives, each dispatchable from the CnC:

- UDP flood (raw and “plain” variants).
- TCP SYN flood.
- TCP ACK flood.
- TCP STOMP – a custom Mirai attack designed to bypass certain DDoS mitigations.
- GRE flood (Generic Routing Encapsulation) – novel for the time; many mitigation services didn’t handle GRE traffic.
- DNS water-torture attack – a DNS amplification variant.
- HTTP flood.

Each is implemented in a separate `attack_*.c` file in the source tree. They’re not subtle; they’re just very fast packet generators that scale across hundreds of thousands of infected devices.

3.7.6 Reading the Mirai source

The Mirai source is on GitHub in multiple mirrors. Search “mirai-source-code” to find them. For deep understanding, read in this order:

1. `loader/` – the loader that infects new devices. Around 1500 lines. The multi-method binary delivery (`wget` / `tftp` / `echo`) is fascinating engineering.
2. `bot/scanner.c` – the Telnet brute-force scanner. 600 lines. See the IP exclusion list, the credential list, the per-connection state machine.

3. `bot/killer.c` – the competing-malware killer. 300 lines. See the `/proc` enumeration and process-matching logic.
4. `bot/attack.c` and `bot/attack_*.c` – the DDoS implementations. Skim; the packet-construction details aren't essential.
5. `bot/main.c` – the bot's main lifecycle: `ensure_single_instance()`, `rand_init()`, `anti_gdb_entry()`, the calls to start each subsystem, the main loop.
6. `cnc/main.go` – the C2 server, written in Go. Skim. Less important for understanding the bot behavior.

As you read, note for each stage:

- Which syscalls happen at this stage and roughly in what frequency.
- What network behavior is visible.
- What filesystem behavior (usually none for the bot, some for the loader).
- What process operations happen.
- How an ideal detector would catch this stage.

This exercise – mapping each stage to detectable behavior – is one of the best ways to internalize what behavioral detection actually has to work with.

Further reading

- Antonakakis et al. 2017 – Sections 3 (architecture) and 4 (measurement) are required reading.
- The Mirai source code – multiple GitHub mirrors.
- Krebs on Security, “Who is Anna-Senpai” – the author identification and the legal aftermath.
- Antonakakis presenting at USENIX 2017 – video on YouTube.

3.8 Honeypots and Data Collection

3.8.1 What honeypots are

A honeypot is a system pretending to be a vulnerable device, exposed to the Internet specifically to attract attackers. Researchers and defenders use honeypots to:

- Catch new malware samples in the wild as they spread.
- Observe attack techniques: credential lists, exploit attempts, command sequences.
- Measure prevalence – how common is each technique?
- Identify attackers by source IP, timing, technique signatures.
- Build datasets for research.

The classical taxonomy by interaction level:

- **Low-interaction.** Emulate just enough to log the attack attempt; no real shell or full system. Catches the connection attempt but typically not the post-exploitation payload. Cheap to run, safe.

- **Medium-interaction.** Emulate a more complete shell environment. Catches the credential brute force, logs subsequent commands the attacker runs, often captures the downloaded malware binary. Cowrie is the classical example.
- **High-interaction.** A real OS, often in a VM with snapshot/rollback. Captures everything but harder to run safely – if the attacker breaks out, they have real network access. Drakvuf and similar tools fit here.

3.8.2 Cowrie

The most-deployed medium-interaction SSH/Telnet honeypot. Open source, written in Python. Pretends to be a Linux box, accepts SSH/Telnet connections, accepts a configurable set of credentials, then presents a fake shell that logs all interaction.

What Cowrie logs:

- All commands attempted by the attacker.
- All files downloaded via `wget`, `curl`, `tftp` – the actual malware samples!
- Session timing.
- Source IPs.
- Detailed protocol-level information.

Cowrie captures of the early Mirai era are how researchers reconstructed the credential lists and the timing patterns. Many academic IoT-malware datasets started as Cowrie logs.

3.8.3 IoTPOT

A purpose-built IoT honeypot from Yokohama National University researchers (the Yoshioka and Matsumoto group, with international collaborators). Designed specifically to catch IoT-targeting malware. The 2015 USENIX WOOT paper is foundational.

Key innovations over Cowrie:

- Multi-architecture binary execution – when an attacker delivers a MIPS binary, IoTPOT can actually run it to observe its full behavior.
- Targets IoT-specific protocols beyond Telnet/SSH (UPnP, CWMP, others).
- Long-running deployment with longitudinal data spanning years.

3.8.4 Honware (2019)

A more recent firmware-emulating honeypot. Combines firmware emulation (Firmadyne-style) with honeypot data collection. The honeypot *looks* like a real, vulnerable IoT device because it's running real firmware. This catches device-specific exploits that wouldn't fire against a generic Cowrie-style honeypot.

3.8.5 The bias problem in honeypot data

Watch out

Critical for ML evaluation: honeypot data has specific biases that affect any model trained on it.

- **Attacker selection bias.** Honeypots attract attackers who don't fingerprint targets before attacking. More targeted, sophisticated attackers may notice anomalies in honeypot behavior and avoid them entirely. The data captured is over-representative of the less-skilled attacker population.
- **Geographic and provider biases.** A honeypot's location (which ASN, which country) affects what attacks it sees. Different regions have different attacker populations and different targeted threats.
- **Temporal bias.** The attack mix at any time reflects what's currently popular. A 2018 dataset is dominated by Mirai variants; a 2024 dataset has different mix.
- **Benign-class scarcity.** Honeypots only catch attacks. The "benign" side of any classification problem has to come from somewhere else – a real OS, a sandbox running known-good software, etc. The mismatch between benign-source and malicious-source creates evaluation artifacts: classifiers may learn to distinguish "honeypot environment" vs "benign environment" rather than "malicious" vs "benign behavior".

This is the kind of subtle bias the TESSERACT paper (Pendlebury et al., USENIX 2019, covered in Track 4) addresses with rigorous evaluation methodology.

3.8.6 Where to find samples

MalwareBazaar (bazaar.abuse.ch). Free, community-driven. Many IoT/Linux samples. Samples are downloadable for research with appropriate caution.

VirusTotal. The industry standard for sample sharing. Free for low-volume use; academic and researcher access programs exist for bulk access.

vx-underground. A community archive of samples and academic papers.

University-hosted datasets. Some research groups maintain shareable corpora with access controls. Look at recent papers' "data availability" statements.

Mirai source code. On GitHub in multiple mirrors – more educational than a binary because the comments and structure are intact.

Watch out

Always handle real malware samples in isolated environments: a dedicated VM with no network access (or snapshotted to allow rollback), or air-gapped hardware. Do not download samples to your daily-driver machine.

For learning purposes, the Mirai source code is more educational than a compiled binary. Reading well-commented C is much faster than reverse-engineering a stripped binary, and you need no isolation precautions to read source.

Further reading

- Pa, Suzuki, Yoshioka, Matsumoto, Kasama, Rossow, “IoTPOT: Analysing the Rise of IoT Compromises” (USENIX WOOT 2015) – foundational.
- Vetterl, Clayton, “Honware: A Virtual Honeypot Framework for Capturing CPE and IoT Zero Days” (eCrime 2019).
- [Cowrie GitHub](#).
- [MalwareBazaar](#).

3.9 Track 3 Self-Check Questions

1. Describe Mirai’s infection lifecycle in 60 seconds, naming the four stages and what each does.
2. What’s the difference between memory-resident and persistent malware? Why does Mirai choose memory residence? What’s the trade-off?
3. Name three evasion techniques and three behavioral signals that would still detect them.
4. Why is static analysis weaker for IoT malware than for Windows malware? Why is dynamic analysis correspondingly more important?
5. What is the killer routine in Mirai? Why is it a particularly clean behavioral signal?
6. Why does honeypot-collected malware data introduce bias into ML evaluation? Name three specific biases.
7. Trace, in syscalls, what Mirai’s scanner does during a single Telnet brute-force attempt against a single target.
8. What does telfhash compute, and why is it useful for IoT malware specifically?
9. Why does `mprotect(..., PROT_EXEC)` on a non-JIT process serve as a strong injection signal?
10. How does the choice of feature representation (frequency vector vs n-grams vs raw sequence) affect what a syscall-based classifier can learn?

4 Machine Learning for Security

Machine learning is the standard tool for turning behavioral data into detection decisions. But security ML is a notoriously hazardous subfield – a substantial fraction of published work doesn’t replicate, makes claims that don’t generalize, or evaluates on toy problems that don’t reflect deployment realities. The mistakes are remarkably consistent across papers and decades. This track covers both the mechanics (how to actually build classifiers) and the methodology (how to evaluate them honestly).

The most important takeaway from this track is methodological: get the evaluation right and most other choices fall into place. Get it wrong and even sophisticated models produce misleading results.

4.1 The Mindset of ML in Security

4.1.1 Why security ML is different from textbook ML

Standard ML assumes the world is roughly stationary: training data and test data come from the same distribution, the prediction problem doesn’t change shape over time, and the cost of different errors can be summarized by a single metric.

In security, none of these assumptions hold:

Adversaries adapt. If your classifier reliably detects today’s malware, tomorrow’s malware will be different. Attackers actively probe defenses and modify their software to evade detection. The distribution drifts continuously, often in adversarial directions.

Class imbalance is extreme. In any real environment, the rate of malicious events is tiny compared to benign events. A network with 99.99% benign traffic and 0.01% malicious traffic is realistic. This breaks standard metrics: a classifier that says “benign” for everything achieves 99.99% accuracy and zero useful detections.

The costs of FP and FN are very different and very context-dependent. A false positive (flagging benign as malicious) at scale generates alert fatigue and erodes trust in the system. A false negative (missing real malware) can be catastrophic. The relative weight depends on the operational context, the alert volume the SOC can handle, the downstream consequences of each error type.

Ground truth is contested. Is this sample malicious? It might be malware. It might be benign software with intrusive behaviors (adware). It might be a security tool that looks like malware. Labels from different sources disagree. A sample’s label can change over time as more information emerges.

The world is not iid. Different organizations see different attacks. Different time periods see different malware mixes. Different network locations face different threats. A model trained on one organization’s data may not transfer to another.

Evaluation requires temporal discipline. A model trained on 2020 data and tested on 2024 data is the realistic deployment scenario. Most academic evaluations don’t do this; they random-split a single year’s data and test on the held-out portion. The resulting accuracy estimates are optimistically biased.

4.1.2 The base rate problem

This is the single most important concept in security ML and the most commonly mishandled. Stop me if you've heard this one:

A test for a disease has 99% accuracy. If you test positive, what's the probability you have the disease?

The naive answer is 99%. The correct answer depends on the base rate – how common the disease is in the population. If the disease afflicts 1 in 10,000 people, and your test has a 1% false positive rate, then:

- In a population of 10,000, 1 person actually has it.
- 99% test sensitivity means we catch 1 true positive.
- 1% false positive rate on the 9,999 healthy people means 100 false positives.
- Total positives: 101. True positives: 1.
- $P(\text{disease} \mid \text{positive test}) \approx 1/101 \approx 1\%$.

A “99% accurate” test produces 100 false positives for every true positive when the base rate is 1/10,000.

Now translate to security. If 0.01% of network traffic is malicious, and your classifier has 99% accuracy on both classes:

- Out of 1,000,000 events: 100 are malicious.
- 99% true positive rate $\Rightarrow$ 99 true positives.
- 1% false positive rate on the 999,900 benign events $\Rightarrow$ 9,999 false positives.
- For every true detection, the analyst gets 100 false alarms.

At a realistic security base rate, even a 99%-accurate classifier may be unusable because the alert volume is overwhelming. To be deployable, classifiers often need false positive rates well under 0.01%, which is genuinely hard.

Key idea

Always think about base rates. A claim like “99% accuracy on a balanced test set” tells you almost nothing about deployment performance. The metric that matters is the operational *precision* (true positives / total positives), and that depends on the base rate in your actual environment, not on the test-set composition.

When reading security ML papers, always look for: what's the assumed base rate? What's the false positive rate at the chosen operating point? How many false alarms per day would this generate at realistic deployment scale?

4.1.3 Precision, recall, F1, ROC, PR

The standard metrics, with notes on which are appropriate for security:

Accuracy. $(TP + TN) / \text{total}$. Almost useless under class imbalance. Don't use.

Precision. $TP / (TP + FP)$. The fraction of positive predictions that are correct. Critical for security: this is “of all my alerts, how many are real?” A precision of 0.01 means 99 of every 100 alerts are false alarms.

Recall (aka sensitivity, true positive rate). $TP / (TP + FN)$. The fraction of actual positives that you catch. Also critical: “of all the real threats, how many do I detect?” A recall of 0.50 means you miss half of all real threats.

F1 score. The harmonic mean of precision and recall. A single number that balances both. Useful for comparison, but it implicitly weights precision and recall equally, which may not match operational priorities.

False positive rate. $FP / (FP + TN)$. Critical when reporting; should be reported at multiple operating points.

ROC curve. Plot true positive rate vs false positive rate at various threshold settings. AUC (area under the ROC curve) summarizes overall performance.

Warning about ROC under imbalance: the ROC curve can look great even when precision is terrible, because the FPR axis is the fraction of benign samples misclassified – and with millions of benign samples, even a small FPR translates to many false positives.

PR curve. Plot precision vs recall. Much more informative than ROC under heavy imbalance. The PR curve drops sharply when the classifier starts producing too many false positives.

For security: always include the PR curve. The ROC curve alone is misleading under realistic imbalance.

4.1.4 Concept drift

The phenomenon that the relationship between features and labels changes over time. Malware authors release new families, modify existing ones, change tactics. Legitimate software also evolves – new versions of common tools, new programming languages becoming popular, new application categories emerging.

A classifier trained on 2020 data will degrade when deployed in 2024. The rate of degradation depends on the rate of distribution change in your specific environment, but for IoT malware specifically, expect substantial performance loss within months.

Implication for evaluation: you must measure performance on samples from later time periods than your training data. The standard ML practice of random train/test splits within a single dataset is wrong for security: it gives an accuracy estimate that wouldn’t be achieved in deployment.

4.1.5 Adversarial robustness

A separate concept from concept drift. Adversarial robustness asks: how much can the malware author change the sample to evade detection?

Classical adversarial ML uses gradient-based perturbations (add small noise to an image to flip the classifier’s output). In security:

- Attackers don’t have direct gradient access to deployed classifiers in most realistic settings.
- But they have lots of other levers: rewrite the code, change the order of operations, add no-op operations to dilute features, change which libraries are used.

For behavioral classifiers specifically, an attacker can sometimes pad legitimate behavior into a malicious binary to make its feature vector look more benign – “mimicry attacks”. Defending against this is hard; provable robustness is mostly out of reach.

For research purposes, evaluating adversarial robustness should at least include simple feature manipulations: “what if I add 100 random benign syscalls to every trace?” If your classifier collapses, it’s not robust.

4.2 Feature Engineering for Syscall and Behavioral Data

The features you derive from raw traces largely determine what your classifier can learn. The choice is more important than the choice of model.

4.2.1 Frequency vectors

The simplest representation: a vector of length $|S|$, where entry i is the count of syscall i in the trace.

Variants:

- Raw counts.
- Normalized counts (divide by total syscalls in the trace) – removes trace-length effects.
- TF-IDF weighting – emphasizes syscalls that are rare across the corpus but common in specific samples.
- Log-transformed counts – compresses outliers.

Advantages: low-dimensional, fast to extract, robust to small ordering changes, suitable for any classifier.

Limitations: discards all sequence information. A trace of “socket, connect, send, recv, close” looks identical to “close, recv, send, connect, socket” in this representation, despite meaning opposite things.

For your baseline, always start here. If you can’t beat a Random Forest on normalized frequency vectors with something fancier, your fancier approach isn’t worth the complexity.

4.2.2 N-grams

Sequences of n consecutive syscalls. The feature space grows: $|S|^n$ possible n-grams. In practice the realized n-grams are sparse (most combinations never occur), but vectors can still have thousands of non-zero entries.

Choice of n :

- $n = 1$ – equivalent to frequency vectors.
- $n = 2$ – captures immediate transitions, much more information than frequency alone, modest feature growth.
- $n = 3$ to $n = 5$ – the sweet spot for most behavioral problems. Captures local patterns like “socket, connect, close” (a connect attempt) or “open, read, close” (a file read).
- $n > 7$ – diminishing returns, exploding feature space, sparsity becomes a problem.

Forrest’s original work used $n = 6$ as the sweet spot for sendmail; later work generally finds $n = 3$ to $n = 5$ works well for diverse malware.

4.2.3 Sliding-window histograms

Counts of each syscall in fixed-size windows of W consecutive calls. Produces a sequence of frequency vectors rather than a single one. Detects bursts of behavior:

- A window with 95% `connect` calls = active scanning.
- A window with 95% `read` on file FDs = bulk reading.
- A window with rapid alternation of `kill` = killer routine in action.

Can be fed to sequence models, or processed into burstiness features (variance across windows, peak burst values, etc.).

4.2.4 Sequence models on raw traces

Treat the syscall sequence as a sequence and apply sequence-modeling neural networks:

LSTM / GRU. Recurrent networks. Process the sequence step by step, maintaining hidden state. Good for medium-length sequences.

1D CNNs. Convolutional networks treating the sequence as 1D. Capture local patterns (essentially learnable n-grams) more flexibly.

Transformers. Attention-based models. Can handle long-range dependencies that LSTMs struggle with. Becoming standard but require lots of training data.

These can outperform simple n-gram + classical-ML pipelines on some tasks, but:

- Require much more training data.
- Are harder to interpret.
- Are more expensive to train and serve.
- Often don't beat n-gram baselines as much as you'd hope, especially under proper evaluation.

4.2.5 Graph features

Build a graph from the trace – nodes might be processes, files, sockets; edges represent operations. Then extract features:

- Structural: number of nodes/edges, average degree, clustering coefficient.
- Per-process: fan-out (how many other processes this one interacted with), depth in process tree.
- Specific patterns: process A spawns B which connects to IP C – a parent-child-network relationship.

Graph neural networks (GNNs) can learn over these structures directly. Used in production systems like Microsoft's behavior-based detection.

4.2.6 Cross-feature combinations

In practice the best feature representations combine multiple modalities:

- Syscall frequency vector (overall shape).
- Top-N syscall 3-grams (local sequence patterns).
- Network features (destination distribution, success rate).
- Process tree features (fan-out, depth, signal patterns).

- Filesystem features (write counts in specific directories).

A Random Forest or gradient-boosted trees can absorb hundreds of mixed features without complaint. This is often the best starting point.

Try this

A good exercise: take a syscall trace dataset (Stratosphere IPS has some, ADFA-LD is a classic), compute three feature representations (frequency, 3-grams, sliding-window histograms with $W=100$), train a Random Forest on each. Compare. Note the bias-variance trade-off: simpler features train faster and generalize better; richer features fit better but risk overfitting.

4.3 Classifiers

The choice of classifier matters less than the choice of features and evaluation methodology, but here are the main options:

4.3.1 Random Forest

The workhorse of security ML. Ensembles of decision trees, each trained on a bootstrap sample of the data with random feature subsets.

Strengths:

- Handles mixed feature types (numerical, categorical, binary) without preprocessing.
- Robust to feature scale and outliers.
- Fast to train and predict.
- Provides feature importance for interpretation.
- Hard to overfit catastrophically.
- Strong out-of-the-box performance on tabular features.

Weaknesses:

- Doesn't model sequence directly (you need to engineer sequence features as inputs).
- Doesn't extrapolate beyond training distribution.
- Can be large in memory.

For a Random Forest baseline: 100–500 trees, default depth limits, balanced class weights. With scikit-learn, this is ten lines of code. Beat it.

4.3.2 Gradient-boosted trees

XGBoost, LightGBM, CatBoost. Like Random Forest but trees are trained sequentially, each correcting errors of the previous.

Strengths:

- Generally higher accuracy than Random Forest on tabular data.
- Modern implementations are very fast.
- Handle imbalance well with proper hyperparameters.

Weaknesses:

- More hyperparameters to tune.
- Easier to overfit if you tune poorly.
- Still no native sequence handling.

XGBoost or LightGBM with reasonable defaults is usually 1–3% better than Random Forest on tabular security features. Whether that’s worth the added complexity depends.

4.3.3 SVMs

Support vector machines. Once dominant in security ML, now less common.

Strengths:

- Solid theoretical foundation.
- Work well with kernel tricks for non-linear problems.
- Effective for relatively small datasets.

Weaknesses:

- Don’t scale well to large datasets (training time grows superlinearly).
- Slower than tree-based methods.
- Less intuitive feature importance.

If you see SVMs in older security ML papers, they’re often the baseline. For new work, Random Forest or XGBoost is usually preferable.

4.3.4 Neural networks for sequences

For raw syscall sequences:

LSTMs / GRUs. The classic recurrent approach. Process the sequence step-by-step. Effective on medium sequences (hundreds to thousands of steps). Can be slow to train.

1D CNNs. Convolutional networks treating syscalls as a 1D signal. Faster than RNNs, capture local patterns well, often competitive.

Transformers. Attention-based. Strongest on long sequences with long-range dependencies. Require substantial training data and compute.

Embedding layers. Even classical models benefit from learned embeddings – map each syscall to a dense vector (say, dimension 32 or 64), then process sequences of embeddings. Embeddings can be learned end-to-end with the classifier or pretrained on a large unlabeled corpus.

4.3.5 Anomaly detection

When you have lots of benign data and little or no malicious data, supervised classification doesn’t apply. Instead, build a model of “normal” and flag deviations.

Approaches:

- **One-class SVM** – learns a boundary around the normal class.

- **Isolation Forest** – random trees that isolate outliers (which tend to need fewer splits to be isolated). Fast, scales well.
- **Autoencoders** – neural networks trained to reconstruct inputs. Anomalies have high reconstruction error.
- **Density estimation** – fit a distribution to the normal data; low-probability samples are anomalies.
- **Forrest’s n-gram approach** – the original syscall anomaly detector. Build the set of normal n-grams; flag n-grams outside that set.

Anomaly detection has its own pitfalls: the definition of “normal” depends entirely on training data, and any drift in normal behavior produces false alarms. But it’s appropriate for IoT specifically, where each device class has a very narrow legitimate behavior repertoire.

4.4 Evaluation Methodology

This is where most published security ML goes wrong. Get this section right and your work will be far ahead of typical conference papers.

4.4.1 Temporal splits

Train on past data; test on future data. Always. The standard ML practice of random train/test splits is wrong for security because it lets information “leak” from the future into the training set.

Consider: random-splitting a 2020–2024 dataset puts some 2023 samples in training and some 2024 samples in test. The classifier learns features specific to families that emerged in 2023, which were not available at deployment time in 2022. The reported accuracy is for a model that knew the future during training.

Correct: train on 2020–2022, test on 2023 alone, then on 2024 alone. Report performance per time bucket. Watch it degrade.

Better: train on a rolling window, retrain monthly, report ongoing performance. This matches deployment reality.

4.4.2 Spatial separation

In addition to temporal separation, separate by deployment context. If you have data from multiple organizations or multiple IP ranges, hold out entire organizations or ranges for testing. Otherwise you’re measuring how well the classifier interpolates within sites, not how well it generalizes to new sites.

4.4.3 Class proportions

The class balance in your test set should match the deployment environment, not your training environment. If you trained on a balanced 1:1 ratio (common practice) but deployment has 1:10000, evaluating on 1:1 dramatically overstates performance.

For security: report results both ways. “On the held-out balanced set, recall is X and precision is Y. At the realistic 1:10000 deployment ratio, precision drops to Z.”

4.4.4 Hyperparameter tuning

Hyperparameters tuned on the test set make the test set part of training. Use a separate validation set, or proper cross-validation, for hyperparameter selection. Test set is touched only for the final evaluation.

For security ML, the temporal structure also applies here: tune hyperparameters on a temporal validation split (e.g., 2020–2022 train, 2023 validation, 2024 test), not a random split.

4.4.5 Reporting standards

A good security ML evaluation reports:

- Training set composition: time range, size, class balance, source.
- Test set composition: same details, distinct from training.
- Multiple metrics: precision, recall, F1, AUC-PR (preferred over AUC-ROC under imbalance).
- Performance at multiple operating points (different thresholds), not just one.
- Confidence intervals or variance estimates.
- A baseline comparison: simple frequency-feature classifier with default Random Forest.
- For sequence models: comparison with non-sequence baseline.
- For deep learning: comparison with classical ML baseline.
- Concept drift evaluation: performance over time.
- Sensitivity analysis: how much does performance change with feature changes, label noise, training data shrinkage?

If a paper reports a single accuracy number from a random split, it's making one of the basic mistakes the field has known about for decades.

4.4.6 The TESSERACT framework

The Pendlebury et al. 2019 USENIX paper “TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time” formalizes the methodology problems and provides a framework for honest evaluation. Key concepts:

- **Temporal bias** – training data after testing data in time.
- **Spatial bias** – evaluating on the same population (same organizations, same families) as training.
- **The AUT metric** – “Area Under Time”, summarizing performance over time.

TESSERACT showed that several highly-cited papers' performance numbers collapse when re-evaluated with proper temporal and spatial controls. It's the right starting point for thinking about evaluation in any malware-classification project.

Further reading

- Pendlebury, Pierazzi, Jordaney, Kinder, Cavallaro, “TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time” (USENIX Security 2019). **Required reading.**

- [TESSERACT project page](#) with code.

4.5 The Methodological Pathologies of Security ML

4.5.1 The Arp et al. 2022 paper: “Dos and Don’ts of Machine Learning in Computer Security”

A USENIX Security 2022 paper that systematically catalogued the methodological errors in security ML papers. Read this carefully; the patterns are striking.

The paper identifies 10 common pitfalls grouped into four categories:

Data collection and labeling.

- **Sampling bias.** Data collected in a way that doesn’t represent the deployment population. Honeypot data, malware repository data, single-organization data all have this issue.
- **Label inaccuracy.** Labels from antivirus aggregators are noisy; samples can change category over time; the boundary between malicious and benign is fuzzy.

System design and learning.

- **Data snooping.** Information from the test set leaking into training: through feature normalization computed on all data, through hyperparameter tuning on test data, through label-aware preprocessing.
- **Spurious correlations.** The classifier learns features that correlate with the label in the training set but are artifacts of data collection. Famous example: classifiers that learned to detect “the sample came from VirusTotal” rather than “the sample is malicious”.
- **Biased parameter selection.** Tuning many hyperparameters on the test set inflates performance.

Performance evaluation.

- **Inappropriate baselines.** Comparing only to weak baselines makes any new method look strong. Always include the simple-but-strong baseline (Random Forest on basic features for behavioral detection).
- **Inappropriate measures.** Reporting accuracy on imbalanced data; ROC-AUC without PR-AUC; performance at a single threshold rather than across operating points.
- **Base rate fallacy.** Reporting metrics that don’t account for realistic class imbalance.

Deployment and operation.

- **Lab-only evaluation.** Performance in controlled experiments doesn’t transfer to deployment. Concept drift, adversarial adaptation, integration with existing tools all change the picture.
- **Inappropriate threat model.** Assuming a non-adaptive adversary; assuming the deployment environment matches the experimental one; ignoring evasion possibilities.

The paper reviewed 30 prominent security ML papers and found that all 30 exhibited at least one of these pitfalls. Most exhibited several. This isn’t an indictment of individual researchers; it’s evidence that the methodological standards of the field are systematically inadequate.

4.5.2 What to do about it

For your own work, apply the Arp et al. checklist:

1. Is the data representative of the deployment environment? Document the sampling process and any biases.
2. Are labels accurate? Use multiple sources, report disagreement rates, exclude ambiguous samples.
3. Is the train/test split temporal and spatial? Explicitly check.
4. Are simple baselines included? At minimum, Random Forest on frequency features.
5. Are appropriate metrics reported? Precision, recall, F1, AUC-PR. Multiple operating points.
6. Is concept drift tested? Performance over time, not just on the most recent slice.
7. Are there spurious correlations? Look at what features the classifier actually uses; sanity-check.
8. Is the threat model realistic? Acknowledge evasion possibilities; test against simple feature manipulations.

Doing all of this is work, and the resulting numbers are usually lower than what a sloppy evaluation would produce. The right reaction is to trust the honest numbers.

4.5.3 Related: replicability and the security ML reproducibility crisis

Beyond methodology, security ML has a replicability problem. Papers don't release code, don't release data, don't describe enough details to enable replication. A 2024 follow-up survey to the Arp paper found that even when papers seem methodologically sound, the inability to verify them is itself a serious issue.

For your own work: release code and (where possible) data. Document everything someone would need to reproduce. This makes your work more trustworthy and your future self able to build on it.

Further reading

- Arp, Qiring, Pendlebury, Warnecke, Pierazzi, Wressnegger, Cavallaro, Rieck, "Dos and Don'ts of Machine Learning in Computer Security" (USENIX Security 2022). **Required.**
- Pendlebury et al., TESSERACT 2019 (as cited above).
- Sommer, Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection" (IEEE S&P 2010) – a decade-older paper making many of the same points. The fact that the field still needed Arp 2022 to point them out a decade later is telling.

4.6 The Cost-of-Collection Dimension

A dimension often missing from accuracy-focused evaluations: how expensive is it to actually collect the features at deployment time?

For behavioral detection in IoT, this matters enormously:

eBPF tracing of all syscalls. Roughly 1–5% CPU overhead per syscall, depending on what filtering you do. On an IoT device with already-tight resources, this might be too much.

Sampling subsets of syscalls. Trace only a few syscall types (`execve`, `connect`, `open`, `kill`). Cheaper but misses information.

Statistical sampling. Trace 1% of all syscalls. Even cheaper but very noisy.

Periodic sampling. Trace for 1 minute every hour. Catches steady-state behavior but misses brief bursts.

Network-only. Skip syscall tracing entirely; just monitor outbound network. Much cheaper. Less precise.

Resource pressure monitoring. Just watch CPU/memory/network usage at coarse granularity. Cheap, but only catches behavioral anomalies large enough to show up at this resolution.

The trade-off is between collection cost and detection capability. A detector that achieves 95% recall at 1% CPU overhead may be more deployable than a 99%-recall detector at 30% overhead, depending on context.

When you read papers in this space, look for the cost analysis. Most papers report accuracy on a complete trace; few report what happens when you only have partial data due to deployment constraints. The papers that do this honestly are more useful than those that don't.

4.7 Bringing It Together

A reasonable approach to building a behavioral malware detector for IoT, given everything in this guide:

1. **Get the right data.** Real samples (or honeypot data) for malicious; legitimate IoT workloads for benign. Acknowledge sampling biases.
2. **Standardize the collection.** Use eBPF or another low-overhead tracer. Normalize syscall numbers to names for cross-architecture compatibility.
3. **Engineer features at multiple levels.** Frequency vectors, n-grams, plus network and process features. Start simple; add complexity only with evidence.
4. **Train a baseline.** Random Forest or XGBoost on the combined feature set. Get a number.
5. **Try improvements.** Sequence models, attention, graph features. Each must beat the baseline meaningfully or it's discarded.
6. **Evaluate honestly.** Temporal splits. Spatial splits. Realistic class balance. Multiple operating points. Performance over time.
7. **Quantify deployment cost.** Report CPU overhead, memory overhead, data volume. Consider partial-data variants.
8. **Stress-test adversarially.** What happens with simple feature manipulations? With noise added to traces? With behavioral mimicry?
9. **Document everything.** Code, data (where possible), exact methodology. Future you and others will thank present you.

This is not the workflow most published papers follow. It's the workflow that produces results that survive contact with the real world.

4.8 Track 4 Self-Check Questions

1. Why is accuracy a misleading metric for security classifiers? What should you use instead?
2. Explain the base rate fallacy with a concrete example involving IoT malware detection.
3. Why is a temporal train/test split essential for malware classifiers? What goes wrong with random splits?
4. Name three feature representations for syscall traces and describe what each captures vs. discards.
5. Why is Random Forest a strong baseline for security ML? What does it need from you in terms of feature engineering?
6. What is concept drift and how do you measure it in a malware classifier evaluation?
7. List three of the methodological pitfalls from the Arp et al. 2022 paper and give an example of each.
8. What does the TESSERACT framework specifically check for, and how does it improve on standard evaluations?
9. Explain why PR curves are preferred over ROC curves under heavy class imbalance.
10. How does the cost-of-collection dimension change the trade-off between accuracy and deployability for IoT behavioral detection?